EFES Master Node: a Full Custom FPGA plus ESP32 Implementation of the AUSP Acoustic Protocol

Gioele Giunta
s360501

Politecnico di Torino, LM 32 Computer Engineering
Course: Electronics for Embedded Systems

Academic Year 2025 / 2026

Note on this report. I proposed this project to the professor as an alternative to the written exam, and the professor approved the idea. So this is not a lab assignment: it is a personal project that I designed and built on my own, and what follows is the report I am sending to him. For this reason the text is written in the first person. The focus is the master node, that is the FPGA side plus the ESP32 that drives it.

Contents

1 Introduction

The AUSP protocol is a communication protocol that I created for my Bachelor thesis and later submitted to the IEEE IoT Magazine under the title *Design and Experimental Validation of a Low-Cost Acoustic Communication Protocol for Underground Sensor Networks*. The goal of the protocol is to let two devices talk to each other using only very cheap hardware, namely a speaker and a microphone. The protocol sends packets by emitting pairs of superimposed tones: at every step one tone identifies the logical channel (the carrier) and a second tone carries the data symbol.

On the receiver side an FFT moves the microphone signal from the time domain to the frequency domain, so that the pair of active frequencies can be read out as peaks in the spectrum. To keep the detection reliable in the presence of noise I had implemented, in the original thesis, several techniques: a machine learning stage that learns the noise floor and the low frequency roll-off of the cheap microphone (electret capsules lose sensitivity at low frequency), and a parabolic interpolation that refines the position of each spectral peak to better than one bin.

1.1 What this version is

The version documented here is a full custom FPGA implementation of the *master* side of my protocol, paired with an ESP32 in a hybrid arrangement. The FPGA is a Tang Nano 20K (Gowin GW2AR-18). I used Gowin EDA for synthesis, place and route and programming, and GHDL to simulate my VHDL before putting it on the board.

On the FPGA I preferred to write the interconnect, the peripherals and the memory controller myself in VHDL, and to drive their registers from a soft core CPU so that all the high level logic lives in software. I used only three IP cores from Gowin:

The **PicoRV32** soft core is the CPU that runs the firmware and drives every peripheral over the on chip bus. The **Gowin rPLL** takes the 27 MHz board crystal and produces the two synchronous clocks the design needs (40.5 MHz for the bus and the CPU, and a phase shifted copy of it for the SDRAM die). The **Gowin FFT** core sits inside the hardware accelerator that I built around it to turn microphone samples into a spectrum.

Everything else on the FPGA is custom VHDL that I wrote: a generic UART module (instantiated twice, for the protocol characters and for the storage commands), a generic SPI byte engine used to talk to the NOR flash, a second SPI master specialised and timed for receiving audio from the ADC, a third, transmit only SPI that drives a small TFT display, a generic GPIO peripheral, and a PWM peripheral that generates the output tones for the speaker.

On the ESP32 side, the board connects to the FPGA through two independent UART links, so four wires in total (two TX and two RX), accessed at register level. The ESP32 runs the AUSP protocol logic: it queues the inbound and outbound symbols, rebuilds the packets, and decides which packet to send next. The ESP32 is also connected to the internet and exchanges commands with a web dashboard over a WebSocket. In this arrangement the FPGA is the interface that lets the system speak my acoustic protocol, and the ESP32 is the brain that runs the protocol on top of it.

1.2 The hardware I used

For persistent storage I used a W25Q64 NOR flash (64 Mbit, 8 MByte). The local display is a 2 inch IPS TFT, a GMT020-02-7P with the usual ST7789 controller, 240×320, driven over SPI. The microphone is an ordinary electret condenser capsule. Its signal is amplified by an MCP6022 operational amplifier and digitised by an MCP3201 12-bit ADC, which the FPGA reads over SPI. The speaker is an ordinary 8 Ω speaker driven from the PWM output through a low side MOSFET. Power comes from a single 12 V supply that a DFRobot DFR1202 buck module steps down to 5 V for the Tang Nano `VIN`; the 12 V rail also directly feeds the speaker stage. Several `IRLZ44N`/`IRLZ44NPBF` MOSFETs do the switching, plus the usual capacitors and diodes.

The slave device, the one that receives orders over the air from this master, is much simpler and is documented elsewhere: it is an ESP32 wired at register level to an I2S microphone on one side and, through I2S again, to a class D amplifier and a speaker on the other.

2 System overview

The system is a two node star. The master is the hub, the slave is the single spoke, and a web dashboard opened in any browser is an optional viewer. Figure 1 shows the picture.

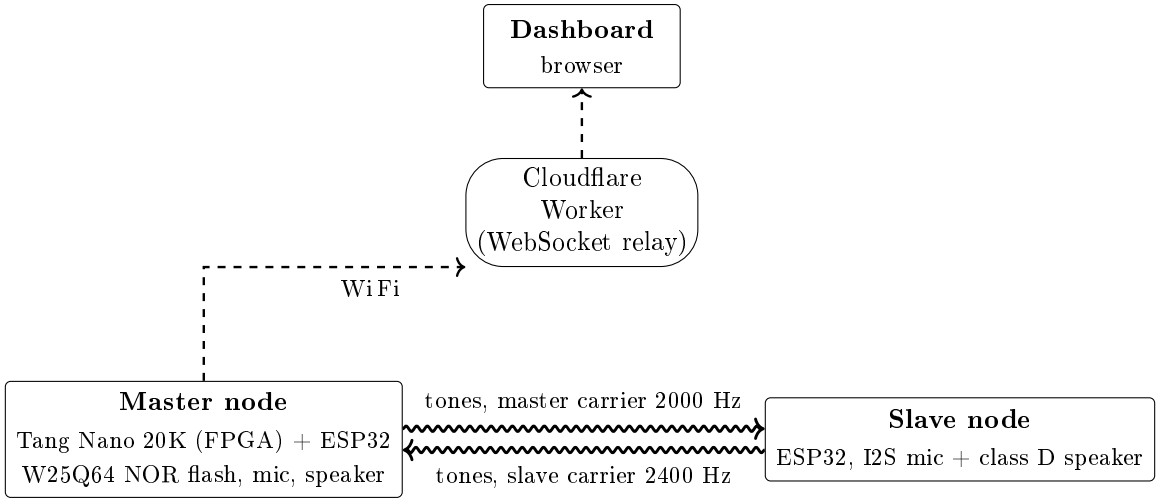

Figure 1: System level view. The only link between master and slave is acoustic (wavy arrows). The WiFi link only carries dashboard traffic; the slave does not need it.

The acoustic link is the only path between master and slave. There is no shared cable, no Bluetooth, no I2C. Everything the two say to each other, presence requests, identification, acknowledgements, abort, has to be encoded into tones and demodulated on the other side (Section 14). The WiFi link is only there so I can watch and command the system from a browser; it is not part of the acoustic protocol.

Inside the master, three units split the work: the FPGA fabric, the soft core (a PicoRV32 inside the FPGA), and the ESP32. The fabric does everything that is timing critical or sample rate bound: it reads the ADC, computes the FFT, copies the spectrum into an on chip snapshot, generates the output PWM tone, and shifts the raw SPI frames to the NOR flash and the TFT. The soft core does the per frame work that is not sample bound: it reads the spectrum snapshot over the bus, decodes the tones, decides whether the channel is free before emitting, runs the W25Q storage protocol and draws the display. The ESP32 does everything that is decision bound: it runs the protocol state machine, talks to the dashboard, and tells the soft core which tones to emit. Table 1 lists the split operation by operation. The units are joined by two UARTs, one carrying the decoded and to be emitted acoustic symbols, the other carrying the flash storage commands.

Table 1: Hardware/software partition: who performs each operation. *Fabric* is fixed FPGA logic, *soft core* is C running on the on chip PicoRV32, *ESP32* is the companion microcontroller. The fabric only samples, transforms, moves, emits and stores; every decision (decode, channel sense, protocol) runs in software.

Operation	**Where**	**Type**
ADC sampling (MCP3201, SPI)	FPGA fabric	hardware
Move samples to SDRAM (DMA)	FPGA fabric	hardware
512 point FFT	FPGA fabric	hardware
Store spectrum in SDRAM	FPGA fabric	hardware
Copy spectrum to BSRAM snapshot	FPGA fabric	hardware
Two tone generation (DDS, sine sum)	FPGA fabric	hardware
NOR flash storage (W25Q protocol)	soft core	software
TFT dashboard rendering	soft core	software
Read snapshot over the Wishbone bus	soft core	software
Tone decode (carrier + bit)	soft core	software
Carrier sense, emit gating (CSMA)	soft core	software
Choose which tones to emit	soft core	software
Protocol state machine	ESP32	software
Pattern ↔ message framing	ESP32, soft core	software
WiFi and dashboard	ESP32	software

2.1 The board around the FPGA

The SoC of the next sections lives *inside* the FPGA, but the master node is a whole board: a power chain that ends in the FPGA, and a ring of real parts around it (storage, display, indicator LEDs, the analog microphone front end and the speaker driver). Figure 2 is the board level schematic with the actual components I used. Three supply rails organise it. The 12 V comes from an Apple USB-C charger through a USB-C *trigger* module (it negotiates a fixed 12 V over PD/QC/AFC, so the charger behaves like a plain 12 V brick); that 12 V feeds both a DFR1202 buck converter, which steps it down to the 5 V the Tang Nano 20K wants on `VIN`, and the speaker, which is loud enough to run straight off 12 V. The FPGA board then exposes two rails of its own: **3.3 V** for the digital peripherals and **5 V** (the `VIN` passed through) for the analog front end. One detail that bit me and is now load bearing: the 3.3 V does not go straight to the peripherals, it passes through an `IRLZ44N` MOSFET that the GPIO (pin 80) holds on, so the GPIO output *must* stay at `1` or the NOR flash, the display and the LEDs all lose their supply (Section 3).

3 The FPGA SoC

The Gowin GW2AR-18 holds the custom SoC. It is a 20k LUT class FPGA, and the AR variant has an 8 MByte SDR SDRAM die inside the same package, reachable on dedicated fabric pins (no external SDRAM soldering). Figure 3 is the top level block diagram.

3.1 Block diagram

3.2 The on chip bus and the address map

The interconnect (`wb_interconnect.vhd`) is a shared bus crossbar with **two masters and ten slaves**, all live: the two ports that earlier revisions kept reserved for the flash migration (S7, S8) now carry it (Section 9), and a tenth port (S9) was added for the TFT display (Section 10). The two masters are the PicoRV32 CPU (port M0) and the audio accelerator (port M1). When both request the bus in the same cycle, M1 wins: the accelerator is sample rate bound and must not

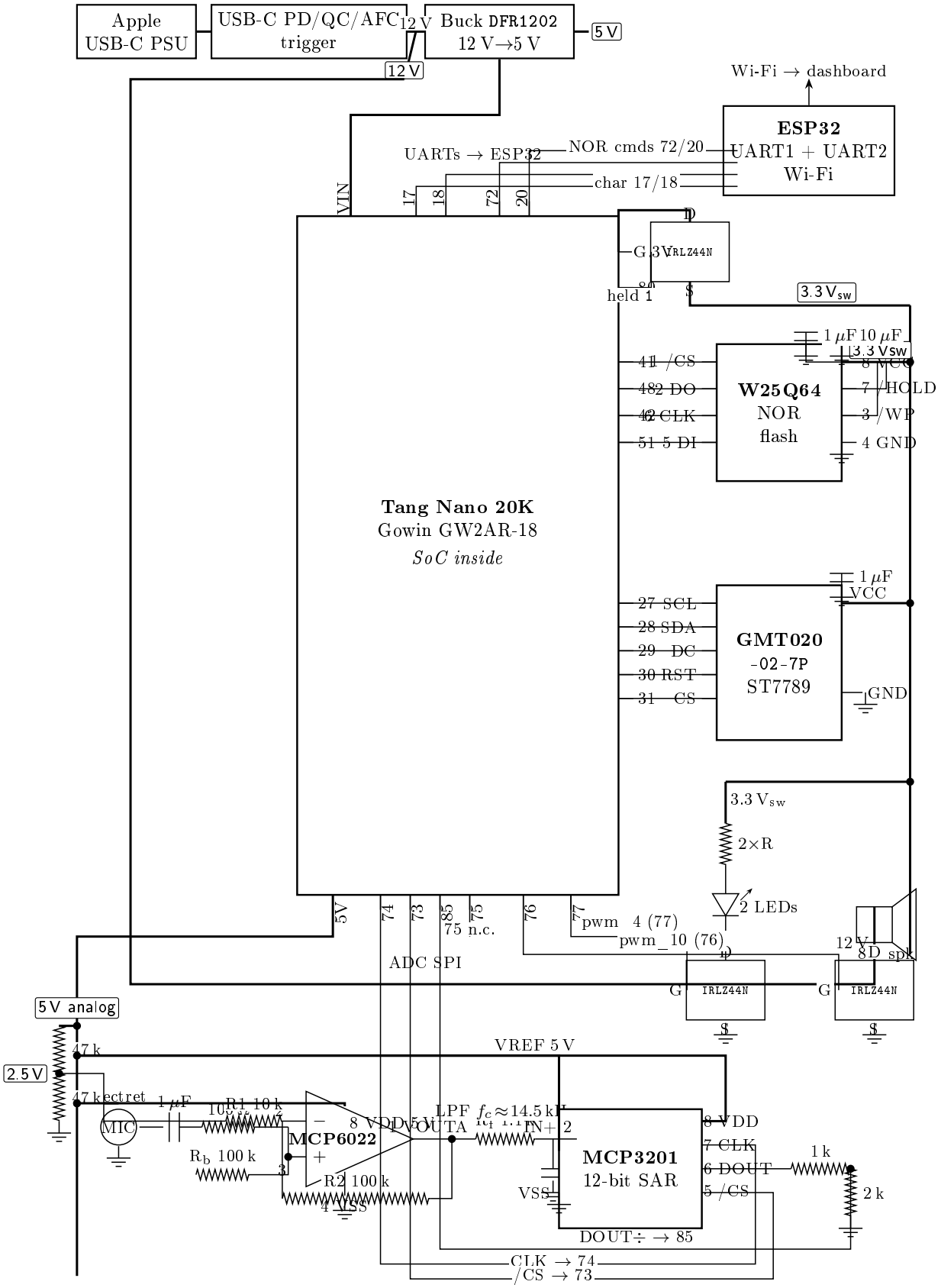
Apple
USB-C PSU
USB-C PD/QC/AFC
trigger
12 V
Buck DFR1202
12 V→5 V
5 V
12 V
Wi-Fi → dashboard
ESP32
UART1 + UART2
Wi-Fi
UARTs → ESP32
NOR cmds 72/20
char 17/18
17
18
72
20
VIN
G
3.3 V
IRLZ44N
held 1
3.3 Vsw
1 µF
10 µF
3.3 Vsw
/CS
DO
CLK
DI
W25Q64
NOR
flash
8 VCC
7 /HOLD
3 /WP
4 GND
Tang Nano 20K
Gowin GW2AR-18
SoC inside
1 µF
VCC
27 SCL
28 SDA
29 DC
30 RST
31 CS
GMT020
-02-7P
ST7789
GND
3.3 Vsw
2×R
2 LEDs
5V
74
73
85
75
76
77
75 n.c.
pwm 4 (77)
pwm_10 (76)
12 V
8Ω spk
ADC SPI
IRLZ44N
IRLZ44N
5 V analog
47 k
2.5 V
VREF 5 V
47 k
ectret
1 µF
MIC
10 µF
R1 10 k
MCP6022
8 VDD
OUTA
R2 100 k
R_b 100 k
LPF f_c ≈ 14.5 kHz
IN+
MCP3201
12-bit SAR
8 VDD
7 CLK
6 DOUT
5 /CS
VSS
1 k
2 k
DOUT ÷ → 85
CLK → 74
/CS → 73

Figure 2

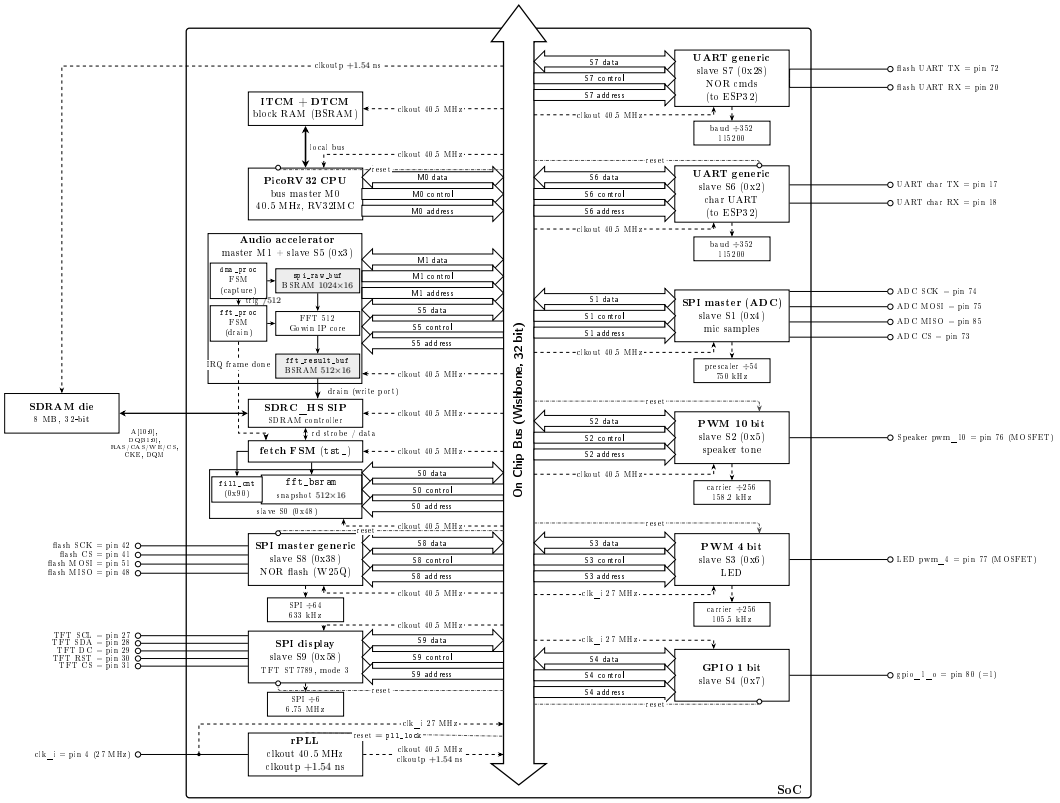

Figure 3: SoC top level (black and white). Each block reaches the central 32 bit Wishbone bus with three thick arrows: *data* is bidirectional, while *control* and *address* arrows point toward the bus for the two masters (PicoRV32, audio accelerator) and toward the block for the slaves, indicating signal direction. The audio accelerator is both a master (M1, to read the ADC) and a slave (S5, for its control registers), so it reaches the bus with *two* sets of arrows: the M1 set drawn in the master direction and the S5 set drawn in the slave direction, like every other slave. Inside it the real modules of `dma.vhd` are drawn: `dma_proc` (the M1 master) filling the sample BSRAM, `fft_proc` feeding the FFT core and collecting into the result BSRAM. Its spectrum *drain* does not cross the bus at all, it owns a dedicated write port of the SDRAM controller, which is why the solid drain arrow enters the *SIP* block directly. The SDRAM port is drawn as its real pieces: the SDRC_HS SIP (the controller that speaks to the die and owns the write port), the *fetch FSM* (`tst_` in the RTL), and the two things the bus actually sees as slave S0, the BSRAM snapshot and the `fill_cnt` frame counter. The dashed arrow is the frame done IRQ: in hardware it starts the fetch FSM, which reads the new spectrum back through the SIP, fills the BSRAM and finally bumps `fill_cnt`; the same line also reaches the PicoRV32 as `irq[20]`, but the firmware ignores it and polls `fill_cnt` over the bus instead (Section 5). The thin dashed nets are the clocks, drawn as a distribution tree rooted in the bus column to keep the routing readable: at the bottom the rPLL injects `clkout` (40.5 MHz) and `clkoutp` (+1.54 ns) into the spine, and the raw 27 MHz crystal, tapped before the PLL (the dot on the `clk_i` wire), enters next to it; each block then receives its own labelled branch, `clkout` for all the SoC logic, `clk_i` (27 MHz) for the LED PWM and the GPIO, and `clkoutp` only for the SDRAM die, leaving the SoC at the top left. The dash dotted net labelled *reset* is the single system reset, the rPLL `lock` (`pll_lock`) injected under the bus like the clocks, so the SoC leaves reset only when the PLL locks; a branch ends in a *circle* where the reset input is active low (as on the CPU `resetn`, the SDRAM controller `rst_n`, and `rst_i` tested against `'0'` on every UART, SPI and the GPIO) and in a *filled arrow* where it is active high (the two PWMs, which take `not pll_lock`). To keep the figure readable the net is drawn to one representative block of each kind; the SDRAM controller shares it too, while the ADC SPI (tied to "never reset") and the accelerator (no synchronous reset) stay unmarked. ITCM and DTCM are CPU local block RAM, not on the Wishbone. The character UART generic slave S6 (`0x2`) reaches the ESP32 on pins 17 and 18 and carries the protocol characters plus the `$` diagnostic lines; the pin is driven by this UART alone (the legacy boot banner and hardware decoder drivers are detached). The **NOR flash** path now runs through the CPU: the storage commands from the ESP32 land on the S7 UART (pins 72/20, top right), the firmware executes the W25Q sequences on the S8 generic SPI engine (pins 42, 41, 51, 48, left), and the old `flash_ctrl` hardware block is retired (Section 9). The **SPI display** slave S9 (`spi_display.vhd`) drives the ST7789 TFT on

be stalled, while the CPU can wait a cycle. The slave that answers a transaction is chosen by decoding the top five address bits `addr[31:27]`.

Five bits, not four, because of where the soft core keeps its own memory. The IP treats every address with `addr[31:29]=000` (`0x0`–`0x1FFF_FFFF`) as *internal* (its ITCM, DTCM and native UART): an external read there never completes, it just stalls the core. The SDRAM slave therefore lives at `0x4800_0000` (`addr[31:29]=010`) and not at `0x1000_0000`; since that base falls on a half nibble, with bit 27 set, the decoder needs the fifth bit to tell it apart from the ADC SPI at `0x4000_0000`. The NOR flash reaches the bus through two slaves with the CPU in between: S7 is the UART that receives the storage commands from the ESP32 (pins 72 and 20), S8 is the SPI engine that drives the W25Q (pins 42, 41, 51, 48); the firmware turns one into the other (Section 9). Table 2 is the real map from the RTL.

Table 2: Address map, decoded from `addr[31:27]` (five bits). All ten slaves are live; the NOR flash is reached through two of them (S7 carries the storage commands from the ESP32, S8 is the SPI engine that drives the chip), with the firmware in between. SDRAM sits at `0x4800_0000`, not in the `0x0`–`0x1` range, because that range is the soft core's own internal space (see text).

Base	`addr[31:27]`	**Slave**	**Block**
`0x4800_0000`	`01001`	S0	SDRAM fetch FSM + BSRAM snapshot
`0x2000_0000`	`00100`	S6	UART generic (characters, pins 17/18)
`0x2800_0000`	`00101`	S7	UART generic (NOR commands, pins 72/20)
`0x3000_0000`	`00110`	S5	Audio accelerator control registers
`0x3800_0000`	`00111`	S8	SPI master generic (NOR chip W25Q)
`0x4000_0000`	`01000`	S1	SPI master (reads the ADC)
`0x5000_0000`	`01010`	S2	PWM 10 bit (speaker tone)
`0x5800_0000`	`01011`	S9	SPI display (TFT ST7789)
`0x6000_0000`	`01100`	S3	PWM 4 bit (LED)
`0x7000_0000`	`01110`	S4	GPIO 1 bit

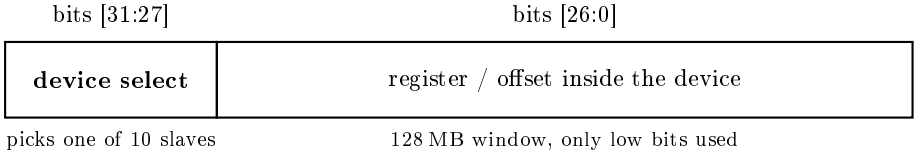

Figure 4: The 32 bit address is split in two. The top five bits `addr[31:27]` pick the slave (Table 2); the low 27 bits are the offset inside that slave, of which each peripheral decodes only the few it needs (the per device register maps below).

A transaction works the standard Wishbone way. The active master drives `adr`, `dat`, `we`, `sel` and asserts `cyc` and `stb`. The interconnect broadcasts the address and write data to all slaves but only routes `stb` and `cyc` to the one slave selected by `addr[31:27]`; the read data and the `ack` are multiplexed back from that same slave. A peripheral therefore needs to do nothing more than raise `ack` for one cycle when it is addressed. The decode is purely combinational on five bits, and any address that matches none of the ten windows falls to a sink that returns zero with no ack.

3.3 Arbitration

There are two masters on the bus, so it needs an arbiter. I used static priority, not round robin: M1 (the audio accelerator) always wins over M0 (the CPU). In the RTL this is one line, the select signal `sel_m1 = m1_cyc AND m1_stb`; when it is high the mux puts M1's `adr/dat/we/sel/cyc/stb` on the bus, otherwise M0's. The `ack` is steered the same way: `m0_ack = slv_ack AND NOT sel_m1`

and `m1_ack = slv_ack AND sel_m1`. So while M1 holds the bus, M0 simply receives no `ack` and stalls on its current access until M1 releases, which is the standard Wishbone way to make a master wait, with no extra logic.

The reason for the fixed M1 over M0 priority is timing. The accelerator is sample rate bound: it must collect every ADC conversion the moment it is ready, and it cannot be made to wait. The CPU has no hard deadline, so it is the one that yields. The cost to the CPU is small in practice, because M1's bus traffic is tiny: the accelerator only uses the bus to service the ADC, one read of the sample register plus one write to clear the ready flag per conversion, a handful of cycles out of the 864 that each 21.3 μs conversion lasts (Section 6), so the CPU is blocked for well under 1 % of the time and never for long enough to matter. The heavy transfer, draining the 512 spectrum words to SDRAM, does not cross this bus at all: the accelerator owns a dedicated write port on the SDRAM controller (Section 5), so the drain and the CPU never compete.

3.4 Three rules the OPEN WB taught the slaves

The PicoRV32's external bus (Gowin calls it OPEN WB) is undocumented at the cycle level, and the slaves in this design went through three rounds of bench debugging before converging on a house style. The three rules below are now applied to every slave, and the newest peripherals (the NOR SPI engine and the display SPI) were born with them already in place.

Hold the ack while the strobe is up. My slaves originally pulsed `ack` for exactly one cycle, which is legal Wishbone. The OPEN WB master misses that pulse often enough that a *read* hangs forever (writes are posted, so they never noticed). The fix is to keep `ack` asserted for as long as `cyc & stb` is, while still edge gating the *side effects* of a write on the strobe's rising edge so a held strobe does not re-trigger them. The GPIO and the two PWMs were retrofitted this way; before the fix, reading them froze the CPU.

Never clear on read. A register that self-clears when read assumes the read that cleared it was also delivered intact, and on this bus that is not a safe assumption. It bit twice: the character UART's `rx_valid` (Section 5.2 tells the same story for `fill_cnt`), and the SPI engine's `done` flag, which a dirty STATUS read would consume, leaving the firmware polling for a completion that had already happened. Both are now non destructive: `done` stays set until the *next* TXDATA write, and status registers can be read any number of times.

Put a FIFO behind anything that streams. The character UART held its last received byte in a single register; a multi byte burst (the 16 byte NOR append payload is the worst case) overwrote it while the CPU was busy elsewhere. The receive side now has a 16 entry FIFO; the read of the data register pops one byte, with the popped value captured into a holding register on the first cycle of the access, because the master samples the data a cycle after the pointer has already moved.

3.5 The simple slaves: UART and GPIO register maps

The two slaves without a section of their own are the generic UART (S6) and the GPIO (S4). The UART is the widest register map in the design; the GPIO is a single bit.

Table 3: UART generic registers (base `0x2000_0000`) and GPIO (base `0x7000_0000`).

Device	Offset	Dir	Function
UART	`0x00`	W	transmit a byte (`dat[7:0]`)
UART	`0x00`	R	`bit[8]`=rx_valid, `bit[7:0]`=received byte (read clears valid)
UART	`0x04`	W	enable
UART	`0x08`	W	disable
UART	`0x0C`	W	`baud_div` = clk / baud
UART	`0x10`	W	format: `dat[1:0]`=parity, `dat[2]`=stop bits, `dat[6:3]`=data bits−5
UART	`0x14`	R	`bit[1]`=rx_valid, `bit[0]`=tx_busy
GPIO	`0x00`	W	`dat[0]` drives the output pin

4 Clock tree

A single 27 MHz clock enters the FPGA on pin 4 (`clk_i`). The active rPLL (`gowin_rpll.vhd`) is configured `IDIV_SEL`= 1, `FBDIV_SEL`= 2, so it divides by 2 and multiplies by 3:

$$f_{\text{clkout}} = 27\,\text{MHz} \times \frac{3}{2} = 40.5\,\text{MHz}.$$

The second output `clkoutp` is the same 40.5 MHz but phase shifted; with `PSDA_SEL`= 1 and `ODIV_SEL`= 16 the shift is one of 16 steps, $360°/16 = 22.5°$, which at 40.5 MHz (period 24.7 ns) is $24.7\,\text{ns} \times 22.5/360 = 1.54\,\text{ns}$.

There are three clocks, and which block runs on which matters. The 27 MHz (`clk_i`) is the PLL reference and also directly clocks the PWM LED and the GPIO. The 40.5 MHz (`clkout`) is the main clock: CPU, Wishbone interconnect, SPI for the ADC, audio accelerator, SDRAM controller, flash controller and the UARTs. The speaker PWM also runs on the 40.5 MHz clock, deliberately: its registers are written by the CPU over the bus, and keeping peripheral and bus in the same domain removes the clock domain crossing on the Wishbone handshake (the LED PWM stays on 27 MHz, since nothing timing critical crosses into it). The 40.5 MHz phase shifted (`clkoutp`) clocks only the SDRAM die, to centre its sampling window (Section 5). Table 4 gives the base clock and the divider each block uses to reach its I/O rate.

Table 4: Base clock and internal divider per block.

Block	Base clk	Divider	Result
SPI for NOR flash	40.5 MHz	÷64	633 kHz SCK, 1 bit = 1.58 μs (slow on purpose, Sec. 11)
SPI for ADC	40.5 MHz	÷54	750 kHz SCK; 16 SCK = 864 cyc/sample → 46.875 kHz
SPI for display	40.5 MHz	÷6	6.75 MHz SCK, mode 3 (TFT ST7789)
UART	40.5 MHz	÷352	115,057 baud (115200 within 0.1 %)
Audio accelerator	40.5 MHz	paced by ADC	one sample per conversion, 46.875 kHz
SDRAM controller	40.5 MHz	none	die clocked by `clkoutp` (+1.54 ns)
PWM speaker	40.5 MHz	DDS phase acc	two tone sine, carrier 40.5M/256 = 158.2 kHz
PWM LED	27 MHz	DDS phase acc	carrier 27M/256 = 105.5 kHz
GPIO	27 MHz	none	static output

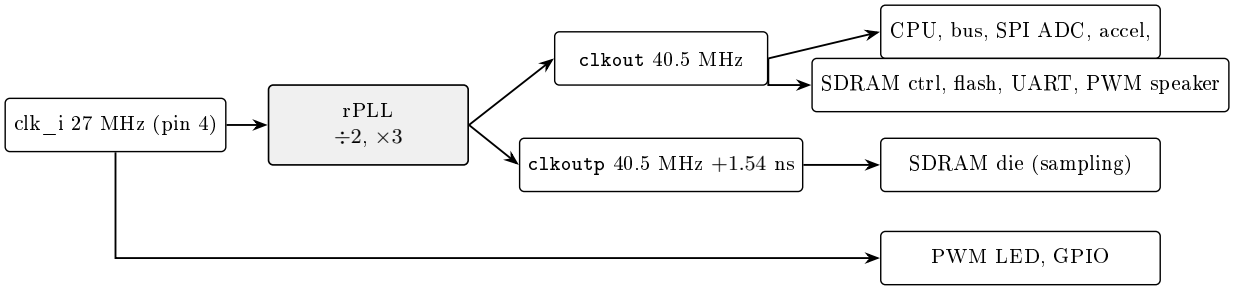

Figure 5: Clock tree. The 27 MHz input is the PLL reference and also clocks the LED PWM and the GPIO directly. The rPLL produces 40.5 MHz (`clkout`) for the CPU, bus and most peripherals, including the speaker PWM (same domain as the bus, so no clock domain crossing on its registers), and a +1.54 ns phase shifted copy (`clkoutp`) that clocks only the SDRAM die.

5 SDRAM controller

The 8 MByte die is internal to the GW2AR-18 package and speaks JEDEC SDR SDRAM. It is not driven directly: the Gowin SDRC_HS controller IP sits in front of it and turns read/write strobes plus a 21 bit {bank, row, column} address into the ACTIVATE / READ / WRITE / PRECHARGE command sequences, and runs the periodic refresh on its own. The IP has a single command port, so in front of *it* there is an ownership mux, which is why the SIP block receives two inputs in Figure 3: while the accelerator asserts `sdr_own` (its drain phase) the write strobe, address and data come from the accelerator; the rest of the time the port belongs to the *fetch FSM*, the read side that serves the CPU (next subsection). The two owners can never collide, because the drain ends, and only then raises the IRQ that starts the fetch. The controller side uses these die pins: `O_sdram_dq[31:0]` (32 bit data), `O_sdram_addr[10:0]`, `O_sdram_ba[1:0]` (4 banks), `O_sdram_dqm[3:0]`, plus CKE / CS / RAS / CAS / WE and the clock. The whole path runs on `clk_sdram` = 40.5 MHz; the die is clocked by `clkoutp`, the +1.54 ns phase shifted copy.

Both sides move data in **bursts of 26 words**, one burst per SDRAM row: the IP is programmed with `data_len` = 26, CAS latency 3, sequential addressing. A 512 word spectrum therefore travels as $20 \times 26 = 520$ word slots (≥ 512; the last eight repeat the final bin and are ignored), each burst going to its own row (rows 2 to 21 of bank 2, all at column 5), written by the accelerator after each FFT and read back row by row by the fetch FSM. Two quirks of this arrangement were found on the bench and are simply worked around rather than fought. First, the very first transaction after a quiet period was unreliable, so each drain starts with a *warm up* burst into row 1 whose content is never read back; the twenty real bursts follow. Second, the IP delivers one extra `rd_valid` pulse per read burst (a 27th word, garbage), which the capture logic discards by gating on index < 26.

I had started, in an earlier version, from single word accesses (burst length 1, the simplest thing that works) and also looked at the opposite extreme, the full page burst, which on paper gives the highest throughput. The 26 word row burst is the middle ground that fits this traffic: the spectrum is contiguous, so a burst amortises the ACTIVATE / PRECHARGE overhead over 26 words, but it is short enough that one burst maps onto one row and the addressing stays trivial (row = burst index, column fixed). One row per burst also means a row is opened exactly once per frame, never reused, so there is nothing to gain from keeping pages open.

5.1 Bring up sequence

The order below was found by testing, not by copying a figure. Each step is the problem I saw and the parameter that fixed it:

1. READ issued right after power up → garbage on every access. The die is not ready yet. Fix: assert `CKE`, wait $\geq 200\,\mu$s for the internal regulator.

2. write then immediate read back → still wrong on the first few accesses, then correct. The refresh counter was not locked. Fix: two AUTO REFRESH commands before programming the Mode Register.

3. PRECHARGE ALL before the refreshes → needed to put all four banks in the idle state first.

So the bring up is: `CKE` high and wait $\geq 200\,\mu$s; PRECHARGE ALL; two AUTO REFRESH; program the Mode Register (CAS latency 3, sequential addressing); after that, ACTIVATE / READ / WRITE / PRECHARGE are allowed.

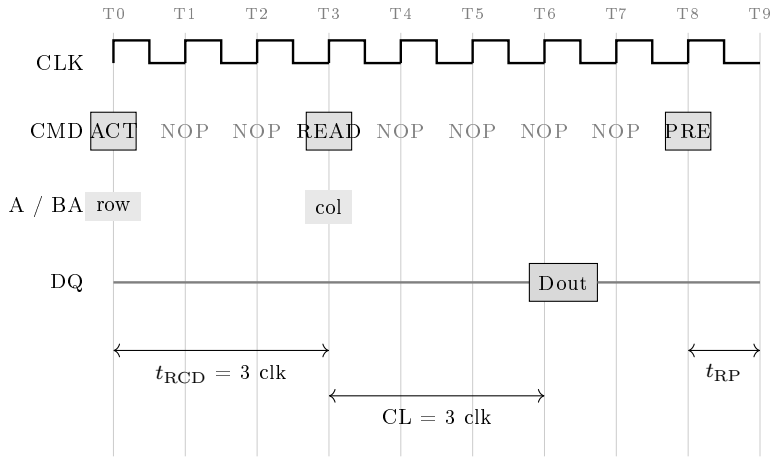

Figure 6: Start of an SDRAM read burst at 40.5 MHz (one clock = 24.7 ns). ACTIVATE at T0 opens the row, t_{RCD} later the READ at T3 selects the column, and with CAS latency 3 the first word is valid on DQ at T6 ($3 \times 24.7 = 74$ ns after the READ). From there the burst continues, one word per clock, until the programmed `data_len` of 26 words is out; then the controller precharges and the row is closed, after which t_{RP} must elapse before the next ACTIVATE (in the figure the burst is cut short to fit the page).

Two more numbers came from the bench. First, skew: at 40.5 MHz the clock period is 24.7 ns, and I was reading `0xFFFE` for `0xFFFF` on a small fraction of accesses, because the latching edge landed at the edge of the data valid window instead of its centre. Driving the die from `clkoutp`, the phase shifted copy, fixed it: with `PSDA_SEL`= 1 and `ODIV_SEL`= 16 the shift is $360°/16 = 22.5° = 1.54$ ns, enough to move the sampling edge into the centre of the window, and the bit errors disappeared. Second, refresh: all 8192 rows must be refreshed every 64 ms, so one AUTO REFRESH every 64 ms/8192 = 7.81 μs, which is 7.81 μs × 40.5 MHz = 316 cycles. My first controller refreshed too rarely and rows lost charge after a few seconds; the 7.81 μs counter made retention solid.

5.2 From the SDRAM to the CPU: the fetch FSM and the BSRAM snapshot

The read side of the SDRAM port is a small state machine that I call the *fetch FSM* (in the RTL its signals still carry the prefix `tst_`: it was born as the SDRAM bring up test machine, then got promoted to production duty and the name stuck; renaming it is on the cleanup list). Its job is to move each freshly written spectrum from the SDRAM to a place the CPU can read calmly.

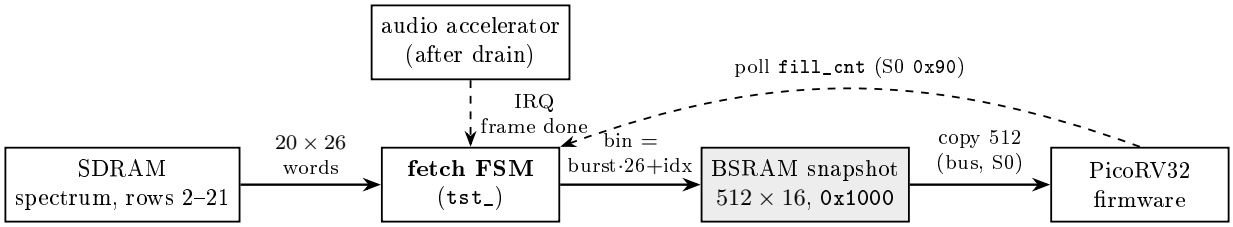

Figure 7: The read path. The accelerator's frame done IRQ does not interrupt the CPU: in hardware it starts the fetch FSM, which reads the 20 row bursts back from the SDRAM and unloads every word into a 512 word BSRAM at its deterministic bin position. When the last burst lands, a frame counter (`fill_cnt`) increments. The CPU polls that counter over the bus and, when it changes, copies the snapshot, which stays frozen for the whole 10.9 ms frame.

The sequence per frame: the accelerator finishes its drain, releases the SDRAM port and pulses the IRQ. The fetch FSM catches the edge, then for each of the 20 bursts it waits for the controller to be idle, issues one read strobe with the row address of that burst, and sits in a fixed 200 cycle window collecting the 26 `rd_valid` words (the 27th, defective, is dropped by the idx < 26 gate). Each word is written into the BSRAM at address burst $\times 26 +$ idx, so the snapshot is filled deterministically, bin by bin. After the last burst the FSM pulses `frame_ready`, the frame counter increments, and the machine parks until the next IRQ. The whole fetch costs about $20 \times 205 \approx 4100$ cycles, $\approx 100\,\mu$s of the 10.9 ms frame.

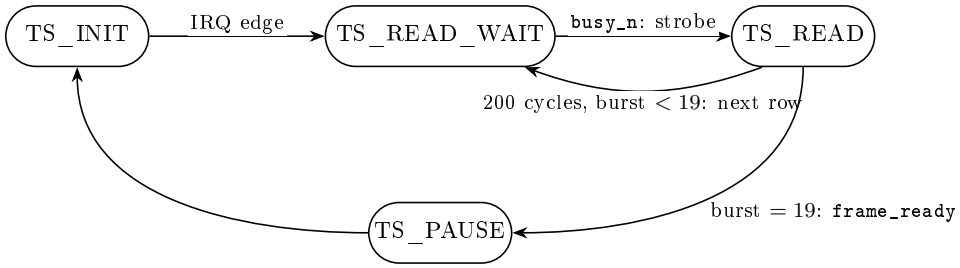

Figure 8: The fetch FSM. One loop iteration per burst: wait for the controller, strobe the read with the burst's row address, collect words for a fixed 200 cycle window while `rd_valid` unloads into the BSRAM, then move to the next row. After burst 19 it signals the frame and re-arms for the next IRQ.

Why the BSRAM in the middle, instead of letting the CPU read the SDRAM words directly? Because the direct version is what I built first, and it produced torn spectra. The first S0 slave exposed the fetch FSM's 26 word burst buffer as bus registers: the CPU was supposed to chase the FSM burst by burst, reading 26 words inside each 200 cycle window. The CPU, which also has protocol work in its loop, regularly lost the race; words from different bursts (and different frames) ended up in the same "spectrum", and the decoder saw peaks that never existed. The BSRAM snapshot removes the race instead of tightening it: hardware fills the buffer deterministically, the buffer holds still for the whole frame, and the CPU copies it whenever it gets around to it within the 10.9 ms. The old burst window is still mapped at offsets `0x00`–`0x64` as a diagnostic.

The same reasoning killed the interrupt. The accelerator's IRQ is physically wired to the PicoRV32 (`irq[20]`), the startup code unmasks it and a full ISR path exists in `crt0.S`; but the interrupt added nothing except a second asynchronous path to debug, since at 91.6 frames per second a polled flag in the main loop is serviced with milliseconds to spare. So the IRQ's production job is to start the fetch FSM in hardware, and the firmware synchronises on `fill_cnt` alone.

`fill_cnt` is a *counter*, not a ready flag, and that is deliberate. A flag must be cleared by the CPU, which means either a write back or a clear-on-read register, and a clear-on-read is exactly

the race that had already bitten me on the UART (the valid bit cleared itself in the same cycle the registered ack sampled it). A counter needs no clearing: the firmware remembers the last value it saw and acts when the value read differs. Missing a frame is harmless by construction, because the comparison still fails on the next poll and the CPU only ever wants the *latest* spectrum; and the snapshot it then copies is guaranteed quiet, because a change of `fill_cnt` means the fill has *just* finished, so the next one is a full 10.9 ms away while the copy takes a few hundred microseconds. Table 5 is the resulting S0 register map.

Table 5: S0 slave register map (base `0x4800_0000`). The snapshot reads are registered BSRAM accesses, acknowledged two cycles after the strobe; everything else acks immediately.

Offset	Dir	Function
`0x00`–`0x64`	R	current 26 word burst buffer (legacy / diagnostic)
`0x80`	R	fetch FSM state: [31] frame ready, [16] in read, [12:8] burst, [5:0] idx
`0x84`	R	counter: ADC samples seen
`0x88`	R	counter: FFT windows triggered
`0x8C`	R	counter: frame IRQs raised
`0x90`	R	`fill_cnt`: completed snapshots (the flag the CPU polls)
`0x1000`–`0x17FC`	R	BSRAM snapshot, bin i at `0x1000`+$4i$ (`xk_re`, 16 bit)

The three hardware counters at `0x84`–`0x8C` exist because, when the chain silently stops, the first question is *where*: if the sample counter moves but the trigger counter does not, the capture side is broken; if triggers move but IRQs do not, the FFT or drain side is. The firmware prints all three once per second on the diagnostic channel (Section 12).

6 Audio accelerator and tone decoder

The receive path is built from two state machines that run in parallel and a ping-pong buffer between them. The accelerator turns microphone samples into a spectrum in SDRAM and raises the frame IRQ; from there the fetch FSM and the BSRAM snapshot of Section 5.2 carry the spectrum to the CPU, which decodes the tones *in software*, consistent with the design goal of keeping the high level logic on the soft core. The CPU starts the accelerator once at boot and never touches a raw sample. The capture FSM (`dma_proc`) reads the ADC over SPI and writes samples into `spi_raw_buf`, a 1024 word block RAM holding two 512 sample windows. The FFT FSM (`fft_proc`) takes the window just filled, feeds it to the Gowin FFT core, collects the 512 spectrum words in `fft_result_buf`, drains them to SDRAM through the dedicated write port and raises the IRQ. While one half of `spi_raw_buf` is being processed, the other half keeps filling, so no sample is lost.

Figure 9 is a zoom into the accelerator drawn in the same visual language as the SoC diagram (Figure 3): the same white blocks, the same shaded block RAMs, the same hollow bus arrows, so it reads as the inset of the `dma.vhd` container blown up to show its real internal modules and its three external interfaces.

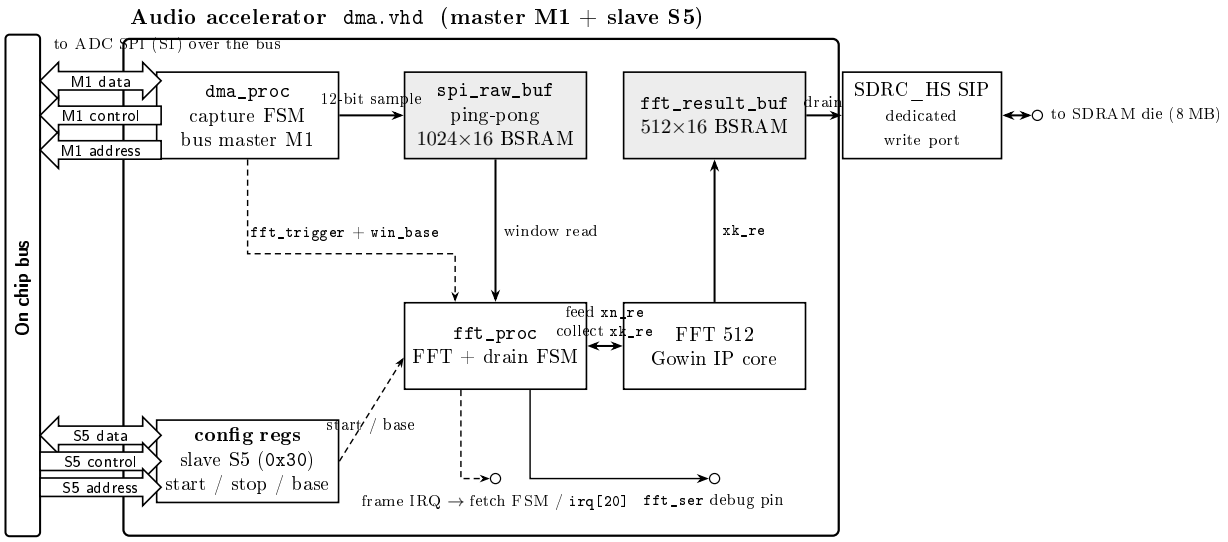

Figure 9: Audio accelerator, zoomed in the SoC style. The container is the `dma.vhd` block of Figure 3 blown up to its real modules. `dma_proc` (the bus master M1) reads the ADC over the Wishbone bus and writes 12-bit samples into the ping-pong `spi_raw_buf`; every 512 samples it pulses `fft_trigger` and hands the just filled window base to `fft_proc`. `fft_proc` reads that window, feeds the Gowin FFT core, collects the 512 spectrum words into `fft_result_buf`, then drains them to SDRAM as 20 row bursts of 26 words through the SDRC_HS SIP's *dedicated write port* (the one path that does not cross the bus, exactly as in Figure 3), and finally raises the frame IRQ. The config registers (slave S5) only carry start/stop and the SDRAM output base; the dashed nets are control, the solid ones are sample/spectrum datapaths. The whole right hand drain never touches the bus, so it cannot collide with the CPU.

6.1 Timing in cycles and in seconds

Everything is paced by the ADC. One SCK period is the prescaler, 54 bus cycles, so $54/40.5\,\text{MHz} = 1.333\,\mu\text{s}$ (SCK = 750 kHz). One MCP3201 conversion is 16 SCK periods, so

$$t_{\text{sample}} = 16 \times 54 = 864 \text{ cycles} = \frac{864}{40.5\,\text{MHz}} = 21.33\,\mu\text{s}, \qquad f_s = \frac{1}{21.33\,\mu\text{s}} = 46.875 \text{ kHz}.$$

One window is 512 samples, so the time to fill it (and the time between IRQs in steady state) is

$$t_{\text{window}} = 16 \times 512 \times 54 = 442{,}368 \text{ cycles} = \frac{442{,}368}{40.5\,\text{MHz}} = 10.92 \text{ ms} \quad \Rightarrow \quad \text{IRQ rate} = 91.6 \text{ Hz}.$$

The processing of one window must fit inside that 10.92 ms. Feeding 512 samples to the FFT, the FFT pipeline, and collecting 512 outputs cost on the order of $3 \times 512 \approx 1500$ cycles at 40.5 MHz, i.e. about 37 μs. The drain itself is cheap: a warm up burst plus 20 bursts of 26 words, each burst about 30 cycles, is ≈ 650 cycles $= 16\,\mu$s. What actually dominates is a deliberate guard: before the first write strobe of every frame the FFT FSM sits in a settle state of 200,000 cycles (4.94 ms), a wait I copied verbatim from the SDRAM bring up procedure that was proven on the bench (the first transaction after a quiet period was the unreliable one, Section 5) and never shrank, because the budget allows it. After the IRQ the FSM also streams the 512 words on a debug pin (8192 cycles, 202 μs). So

$$t_{\text{process}} \approx 0.04 + 4.94 + 0.02 + 0.20 \approx 5.2 \text{ ms} \; < \; t_{\text{window}} = 10.92 \text{ ms} \qquad (\text{margin} \approx 2.1\times)\,.$$

This is the condition that makes the pipeline work: $t_{\text{process}} < t_{\text{window}}$, so processing one window finishes before the next window is full. The ping-pong buffer is what lets the two overlap. The margin is a comfortable 2× rather than a huge one, and almost all of it is the settle guard: trimming that wait is the obvious first move if the budget ever tightens (for example with a larger FFT), but as long as the margin holds, a wait that is known to make the SDRAM reliable is not something I optimise away for elegance.

6.2 The two state machines, drawn as HLSMs

Both machines do real work in every state, not just sequencing, so I draw them as high level state machines: each box keeps the RTL state name and lists the register transfers that state performs, while the edge labels carry the guard and, in braces, the action taken on the transition. The notation is the RTL one: ← is a clocked assignment, `WB` is a Wishbone access on the M1 master, `raw_buf`/`res_buf` are `spi_raw_buf` and `fft_result_buf`.

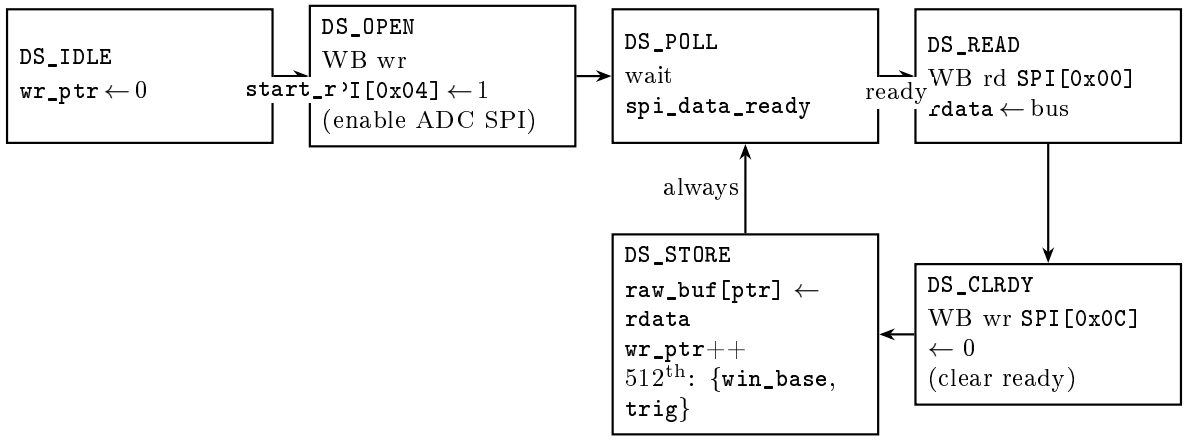

Figure 10: Capture HLSM (`dma_proc`); each Wishbone access advances on its `ack`, left implicit on the edges. After the one time `DS_OPEN` that enables the ADC SPI master, the machine loops `POLL` → `READ` → `CLRDY` → `STORE` once per sample: it waits for the conversion, reads the word over the bus, clears the ready flag, and writes the 12 low bits into the ping-pong buffer. Every 512$^{\text{th}}$ store it also selects the just filled half (`win_base` = 0 or 512) and pulses `fft_trigger`, which is the only thing it ever says to the FFT machine.

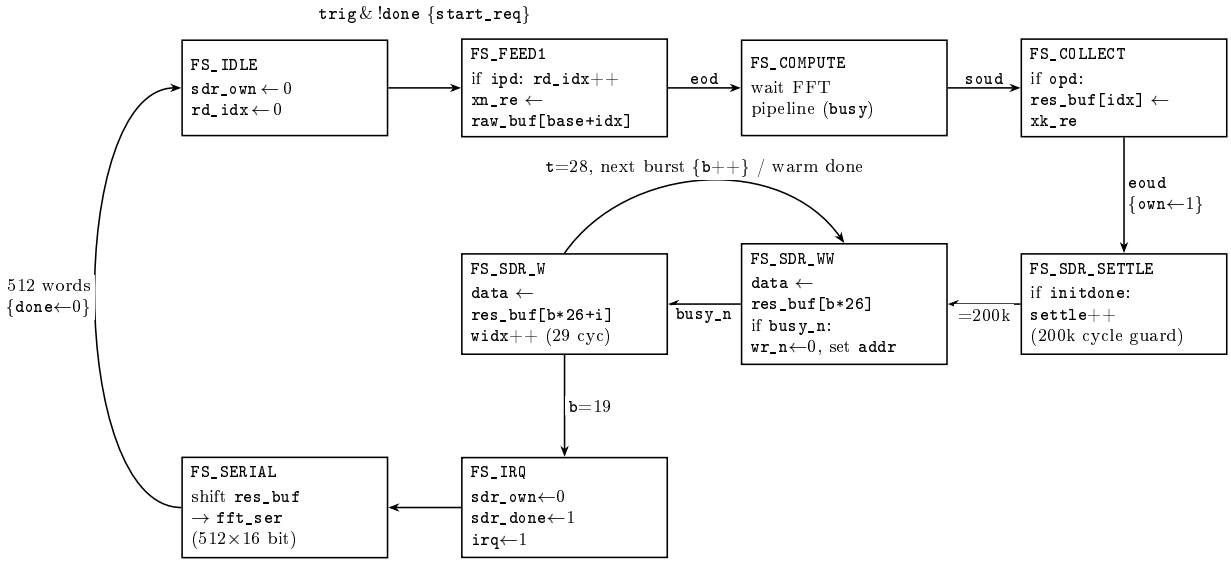

Figure 11: FFT and drain HLSM (`fft_proc`). On a trigger it feeds the window to the Gowin core one sample per `ipd` (`FS_FEED1`), waits for the pipeline (`FS_COMPUTE`), and collects the 512 bins as they come out (`FS_COLLECT`). Then it takes the SDRAM write port and drains: a `200,000` cycle settle guard copied from the proven bring up (`FS_SDR_SETTLE`), then the warm up burst and 20 row bursts of 26 words, each as a `WW` → `W` present-then-strobe pair looping on `b` (`FS_SDR_WW`/`FS_SDR_W`). After the last burst it raises the IRQ and latches `sdr_done` (`FS_IRQ`), streams the frame on a debug pin (`FS_SERIAL`), and re-arms. The collect and drain states touch no Wishbone, so the capture machine on M1 can never interrupt them.

Table 6: Audio accelerator control registers, base `0x3000_0000`.

Offset	Dir	Function
`0x04`	W	start continuous capture + FFT
`0x08`	W	stop
`0x0C`	W	set SDRAM output base: `dat[20:0]` = 21 bit word address

6.3 FFT size and the tone bins

The FFT is 512 point and runs on samples taken at 46.875 kHz, so the bin spacing is

$$\Delta f = \frac{46875 \text{ Hz}}{512} = 91.6 \text{ Hz}.$$

A tone at frequency f lands in bin $k = \text{round}(f/\Delta f)$. The five tones the protocol uses, and the bins the decoder watches, are in Table 7.

Table 7: Protocol tones, their FFT bins, and which node emits them. Master and slave share the same 46.875 kHz / 512 grid ($\Delta f = 91.6$ Hz), so every tone is *on bin* in both directions and a single set of frequencies serves both. The frequencies are the exact bin centres ($f = k\,\Delta f$) used in the code (`emit_tone.c` on the master, `global_parameters.h` on the slave). The two *carriers* are distinct so each side knows who is speaking; the three *data* tones (EOF, bit 0, bit 1) are shared by both directions.

Tone	**Frequency**	**Bin k**	**Emitted by**
master carrier	2014 Hz	22	master only
slave carrier	2472 Hz	27	slave only
EOF marker	2930 Hz	32	both
data bit 0	3571 Hz	39	both
data bit 1	4577 Hz	50	both

The smallest gap between two used bins is between the EOF at bin 32 and bit 0 at bin 39, that is 7 bins or 641 Hz. That gap matters for the decoder, as explained next.

6.4 The tone decoder, with its checks

The decoder ran first as a VHDL block (`audio_decoder.vhd`); when the soft core came up I ported it, check by check, to integer C (`decoder.c`), and the VHDL original stays in the tree as a disabled reference, so the two can be diffed. The checks below are therefore the same in both worlds. For each of the five candidate bins, the decoder decides whether that tone is present. It applies, in order: a local maximum test, the candidate bin magnitude must be greater than or equal to its six neighbours within ± 3 bins ($k-3 \ldots k+3$); a curvature test on the three points $k-1, k, k+1$, which must form a peak that opens downward (positive second difference), so that k is genuinely a crest and not the skirt of a neighbour; and an amplitude test, the parabolically interpolated peak must clear a fixed magnitude threshold. If all three pass, the decoder picks the strongest data tone and the carrier, and emits the character for that pair. The whole test is integer only (no division, no floating point), so it is cheap on the soft core.

The ± 3 neighbourhood, rather than a strict ± 1, is used because the Hann window applied before the FFT spreads a pure tone over two or three bins.

7 SPI master for the ADC, and the microphone front end

The receive chain in front of the FPGA is analog, and getting it described correctly matters; it is the bottom of the board schematic (Figure 2), and the whole chain runs off the FPGA's 5 V rail, not the 3.3 V. The electret microphone produces a few millivolts riding on no particular DC level. It is AC coupled through a $1\,\mu$F capacitor and a $100\,\Omega$ series resistor into an MCP6022 operational amplifier, wired as a non inverting stage, which does two jobs: it biases the signal to mid rail and it amplifies it. The bias comes from a $47\,\text{k}\Omega$ / $47\,\text{k}\Omega$ divider on the 5 V rail, which sets the operating point at $5/2 = 2.5\,\text{V}$, the mid point of the ADC's 0–5 V input range, so the amplified waveform swings symmetrically. The gain is set by the two feedback resistors,

$$G = 1 + \frac{R_2}{R_1} = 1 + \frac{100\,\text{k}\Omega}{10\,\text{k}\Omega} = 11,$$

so a ± 4 mV microphone signal becomes a ± 44 mV swing around 2.5 V, well inside range. After the amplifier an RC low pass acts as an anti aliasing filter before the ADC, with $R_f = 1.1\,\text{k}\Omega$ and $C_f = 10\,\text{nF}$, so the corner is

$$f_c = \frac{1}{2\pi R_f C_f} = \frac{1}{2\pi \cdot 1.1\,\text{k}\Omega \cdot 10\,\text{nF}} \approx 14.5\,\text{kHz}.$$

That corner is chosen to sit comfortably below the Nyquist frequency ($f_s/2 = 23.4\,\text{kHz}$) while passing every protocol tone (the highest is 4577 Hz), so it rejects the higher frequency content that would otherwise alias into the band the decoder watches. The MCP3201 is a 12 bit *SAR* (successive approximation register) ADC: on each conversion it binary searches the input one bit at a time against an internal DAC, and shifts the 12 bit result out over SPI. It is the SPI slave (`Vref` = 5 V), and the FPGA SPI master clocks it to read each conversion. So the chain is

electret mic $\rightarrow 1\,\mu\text{F} + 100\,\Omega \rightarrow$ MCP6022 ($G = 11$, 2.5 V bias) $\rightarrow$ RC anti alias (14.5 kHz)
$\rightarrow$ MCP3201 (12 bit SAR SPI ADC, Vref 5 V) $\rightarrow$ FPGA SPI master.

Table 8: SPI master (ADC) register map, base `0x4000_0000` (offset in the low bits).

Offset	Dir	Function
`0x00`	R	12 bit ADC sample; the read acks only when `data_ready`=1
`0x04`	W	start a conversion
`0x08`	W	stop
`0x0C`	W	clear the `data_ready` flag

7.1 Why a custom SPI master and not a generic one

The MCP3201 has a fixed and slightly awkward frame: after the chip select goes low there is a sample/null period, then the 12 data bits come out most significant first. I wrote a dedicated SPI master for it rather than reuse a generic byte oriented engine, because here I do not send anything (MOSI is unused), I only need to clock 16 SCK periods, throw away the leading null bit, and capture exactly 12 bits into a register, then raise a data ready flag the accelerator polls. Tying the frame to the part lets the sample rate be exact and constant, which the FFT needs.

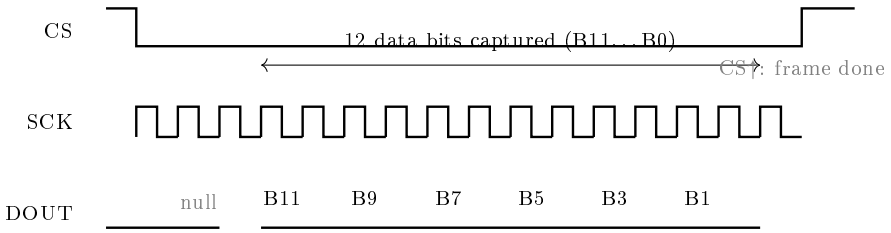

Figure 12: MCP3201 read as seen by the SPI master. CS goes low, the master issues 16 SCK periods at 750 kHz, discards the leading null bit, captures the 12 data bits B11 down to B0 most significant first, then raises CS to end the frame. One frame is $16/750\,\text{kHz} = 21.3\,\mu s$, which sets the 46.9 kHz sample rate.

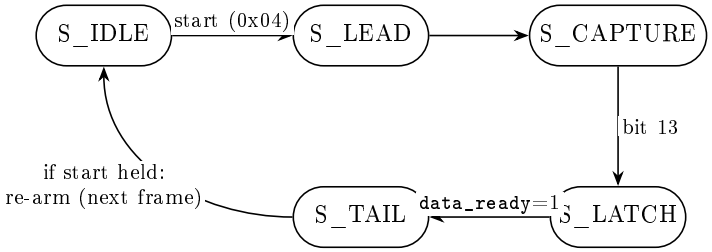

Figure 13: SPI master (ADC) frame FSM, from `spi_master.vhd`. A write to `0x04` asserts CS low and starts one MCP3201 frame of 16 SCK periods at 750 kHz (`PRESCALER`=54 from the 40.5 MHz clock). `S_LEAD` clocks the leading null bit, `S_CAPTURE` shifts MISO in most significant first for the 12 data bits, `S_LATCH` writes them into the `0x00` register and sets `data_ready`, `S_TAIL` raises CS to close the 21.3 μs frame. While `0x04` (start) stays asserted the frame re-arms, which gives the continuous 46.9 kHz sample stream the FFT needs; a write to `0x0C` clears `data_ready`.

8 PWM speaker and LED

Table 9: PWM register map (speaker base `0x5000_0000`, LED base `0x6000_0000`). `F_WIDTH` is the period field width.

Offset	Dir	Function
`0x04`	W	load two tones and start: `dat[F_WIDTH-1:0]`= f_1, `dat[2F_WIDTH-1:F_WIDTH]`= f_2
`0x08`	W	stop

The transmit side is not a square wave generator, even though that would be the simplest thing to do. A square wave at frequency f carries all its odd harmonics $(3f, 5f, 7f, \dots)$ at large amplitude; on the receiver those harmonics land in other FFT bins and can be read as tones that were never sent, and they waste output energy that should sit in the wanted bin. So the block (`pwm_generic_master`) synthesises the tone with direct digital synthesis (DDS) and a sine lookup table, which keeps almost all of the energy at f.

The numbers. The speaker instance runs on the 40.5 MHz bus clock (same domain as the CPU that writes it, no clock domain crossing); the LED instance runs on 27 MHz. A write to `0x04` loads two 16 bit frequencies f_1 and f_2. Each one drives a 24 bit phase accumulator that adds a phase increment every clock,

$$\Delta\phi = \mathrm{round}\left(\frac{f \cdot 2^{24}}{f_{\mathrm{clk}}}\right) = (f \cdot K + 2^{23}) \gg 24, \qquad K = \mathrm{round}\left(\frac{2^{48}}{f_{\mathrm{clk}}}\right),$$

where the right hand form avoids a hardware divide. K is a per instance generic, precomputed because the synthesiser has no `math_real`: 6,949,999 for the speaker at 40.5 MHz, 10,424,999 for the LED at 27 MHz. The top 8 bits of each accumulator index a 256 entry, 8 bit sine table whose entry i is $\mathrm{round}(128 + 127\sin(2\pi i/256))$, centred at 128. The two sine samples are added and halved into one 8 bit duty value, so a single pin carries both tones at once, which is exactly what the protocol needs since it sends superimposed tone pairs.

That 8 bit duty feeds a free running 8 bit PWM comparator, so the carrier is $f_{\mathrm{clk}}/2^8$: 158.2 kHz on the speaker pin, 105.5 kHz on the LED pin, both far above the audio band. The RC low pass on the MOSFET gate then recovers the analog two tone sine and rejects the carrier.

The amplitude is not switched on and off hard either. A hard on/off edge is a step, which is broadband and would add a click plus spectral splatter into the neighbouring bins. Instead an attack/release envelope ramps the AC part of the duty (the swing around the 128 centre) from 0 to full and back over RAMP_MS = 4 ms, one amplitude step every RAMP_TICK =

$(f_{\text{clk}}/1000 \cdot 4)/255$ clocks (≈ 635 at 40.5 MHz). The phase accumulators keep running through the release, so the tone fades out instead of cutting. The envelope is the small FSM in Figure 14.

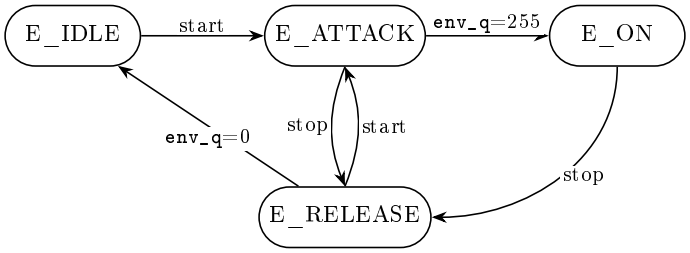

Figure 14: Attack/release envelope FSM in the PWM block. `start` is set by the `0x04` write and cleared by `0x08`. Attack ramps the amplitude up, `E_ON` holds it, release ramps it down; a stop during attack or a start during release switches directly between the two ramps, so a retriggered tone never clicks.

The duty stream drives the gate of a low side MOSFET (an RLZ44N); the speaker sits between the 12 V rail and the MOSFET drain, with a flyback diode across it to absorb the inductive kick when the MOSFET switches off. There is no operational amplifier in the speaker path (the op amp belongs to the microphone input chain, Section 7): the chain is just PWM, RC filter, MOSFET, speaker.

The LED indicator is the 4 bit PWM peripheral, used the same way: its output drives the gate of a second low side MOSFET that switches a 6 W power LED on the 12 V rail, with a series resistor $R = (V_{\text{in}} - V_f)/I_{\text{LED}}$ setting the current. The 4 bits give 16 brightness levels.

Who decides when a tone starts and stops is the firmware, not this block: the soft core's tone player (Section 12) writes the two frequencies to `0x04`, holds the tone for 144 ms, writes `0x08`, and leaves 400 ms of silence before the next symbol, so the receiver has time to finish processing one symbol before the next arrives (a special warm up symbol holds the tone for 300 ms instead, Section 14).

9 NOR flash storage: from hardware block to firmware driver

The flash subsystem has lived two lives. It started as `flash_ctrl.vhd`, the largest VHDL block by line count and the one that caused the most trouble on the bench: a self contained hardware state machine that took high level requests from the ESP32 over a dedicated UART (append a 16 byte record, save the 32 byte settings, read a slot, clear the log) and turned each one into the right sequence of SPI commands to the external W25Q64 chip, with no CPU in the loop. Once the soft core and its bus rules (Section 3.4) proved solid, I retired the hardware block and ported its state machine into firmware (`norflash.c`): the ESP32 still speaks the same one character opcodes, but they now land on a plain UART slave (S7, pins 72/20), the CPU runs the W25Q sequences in C, and the SPI frames go out through the generic byte engine (S8, pins 42/41/51/48). The protocol, the memory map and the SPI sequences below are unchanged between the two lives; what changed is who issues them. The VHDL original stays in the tree as a disabled reference, like the decoder and the emitter before it.

9.1 Memory layout

I use the first 12 KB of the chip, three 4 KB sectors (4 KB is the natural erase granularity of the part). Sector 0 at `0x0000` holds the 32 byte settings: a `0xA5` magic byte, a 15 byte device name, a 2 byte auto presence interval, a 1 byte retry count, and, added with the display, a debug flag byte, the three RGB theme colours (9 bytes), a view flags byte that selects what the TFT zones show, and a second marker byte `0x5A` that distinguishes this extended layout from the old one (a blob without it keeps safe defaults for the new fields). Sector 1 at `0x1000` and sector 2 at `0x2000`

hold the log, 256 slots of 16 bytes each, so 512 slots together, managed as a ring buffer. Each 16 byte record is: 2 bytes sequence number, 1 byte type character (`B` boot, `R` registration, `P` presence, `N` note, `E` event), 4 bytes timestamp, 1 byte value, 8 bytes ASCII text. Figure 15 draws this map.

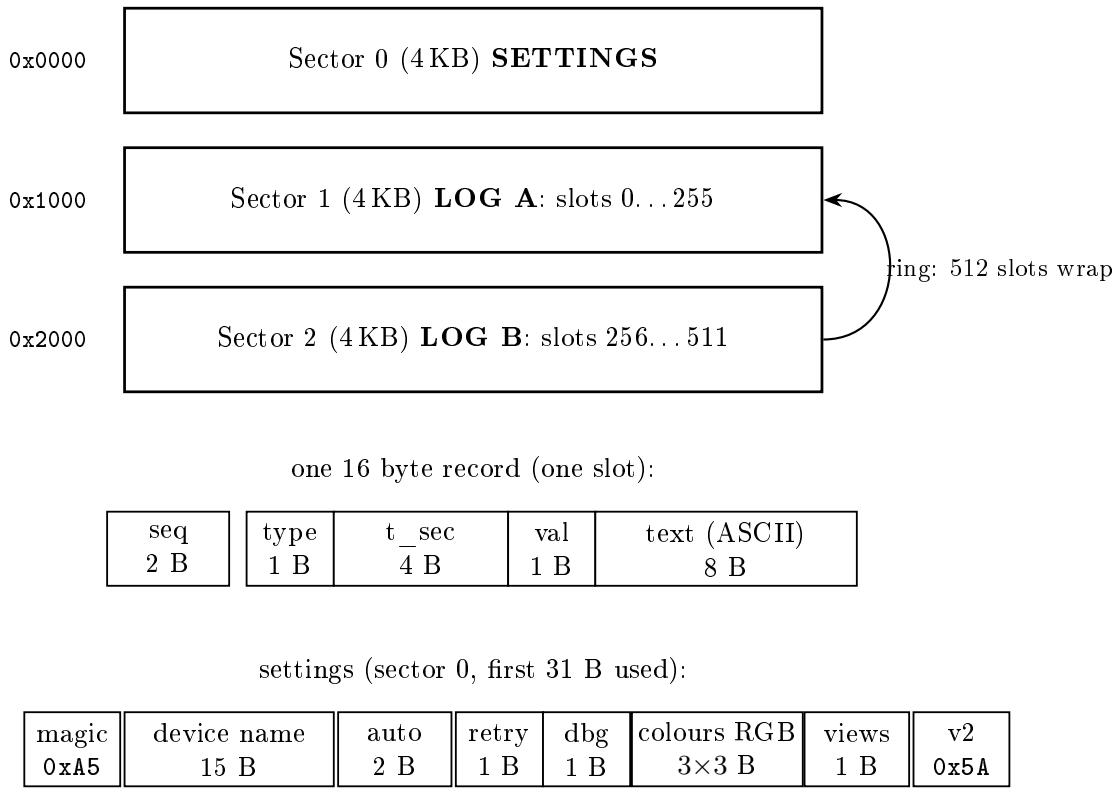

Figure 15: Flash memory map (low 12 KB). Three 4 KB sectors: sector 0 holds the 32 byte settings, sectors 1 and 2 hold the 512 slot log ring buffer. Below, the 16 byte record and the settings sector are cut into their byte fields; the debug flag, the TFT theme colours, the view flags and the `0x5A` v2 marker are the fields added with the display.

9.2 The state machine, and its firmware mirror

Figure 16 is the state machine as it existed in `flash_ctrl.vhd`; the firmware mirrors it almost mechanically. The boot chain on the top row became `nor_init()`, the IDLE hub became the `nor_poll()` call in the main loop (one non blocking peek at the S7 UART per pass), and each opcode handler on the bottom row became a `case` in its dispatch `switch`. Porting a state machine that was already debugged in hardware meant the firmware worked against the real chip almost immediately; the bugs that did appear were all in the bus plumbing, and they are the ones that produced the rules of Section 3.4.

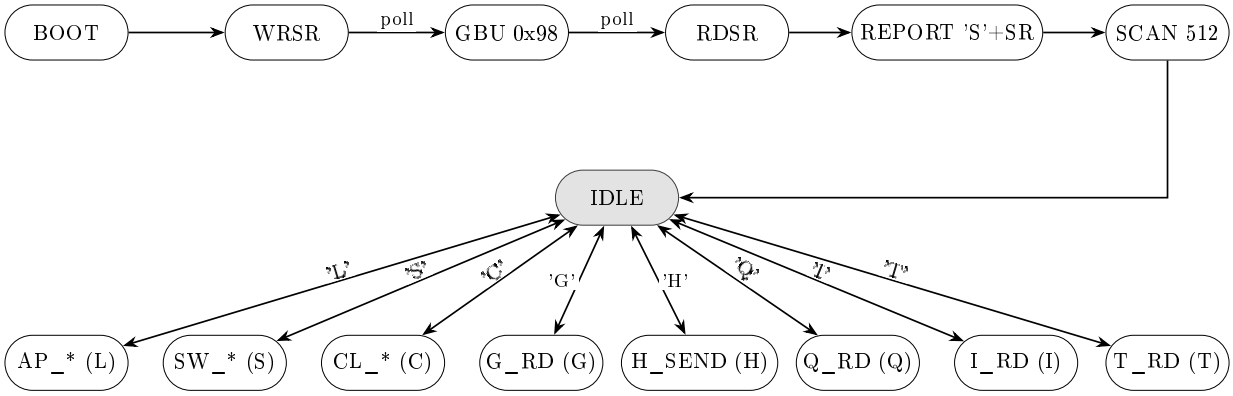

Figure 16: The storage state machine (drawn from `flash_ctrl.vhd`; the firmware port keeps the same shape). At power up it unlocks the chip (WRSR clears the protection bits BP0..BP2 and SRP and sets QE, then a Global Block Unlock), reads the status register back, reports it unsolicited over the UART (`'S'` plus the SR byte; in practice the ESP32 boots too late to catch it and asks again with `T`), and scans all 512 slots to find the head of the ring buffer, then sits in IDLE. Each UART opcode (`L` append, `S` save settings, `C` clear, `G` read slots, `H` read head, `Q` read settings, `I` JEDEC id, `T` read status register) is entered from IDLE on its character and returns to IDLE when its SPI sequence finishes; the double headed arrows mean exactly that go and return.

Append (opcode L) as an SPI sequence. An append is not a single command but a short ordered sequence of SPI frames to the chip, with a status poll between the slow ones. Figure 17 is that sequence in time order.

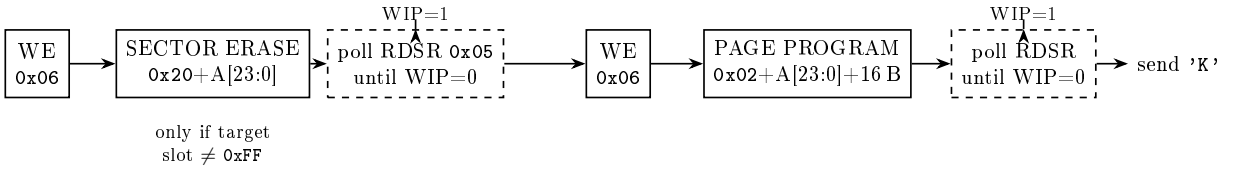

Figure 17: Append (opcode `L`) as the time ordered sequence of SPI frames the controller sends to the W25Q. Each solid box is one CS low frame; the dashed boxes are status polls (read `0x05`, loop while the WIP write in progress bit is 1). The controller first probes the target slot; the erase happens only if that slot is not `0xFF`, because a NOR program can only turn 1 bits into 0, so writing over a used slot would corrupt it. A read (opcode `G`) is instead a single frame: `0x03` + 24 bit address, then clock out $N \times 16$ data bytes.

9.3 SCAN: finding the head after power up

At power up the controller does not know where the last valid record is. It reads byte 2 (the type byte) of every one of the 512 slots and treats a slot as valid only if that byte is one of `B, R, P, N, E`. Anything else (the `0xFF` of an erased slot, or garbage from an aborted write) counts as empty. The head becomes (highest valid slot +1) mod 512. This is more robust than stopping at the first `0xFF`, which I tried first: that version stopped at any hole, even when valid records sat past the hole, which is exactly what a failed retry leaves behind.

9.4 Unlocking the chip, strictly first

The boot order is not negotiable, and the reason is a hardware one. On my board the `/HOLD#` and `/WP#` pins of the W25Q are unconnected, and a floating `/HOLD#` picks up every SCK edge capacitively: the chip pauses itself mid frame, MISO floats, and *every* read returns `0xFF`, including the JEDEC id. So the intuitive plan, "read the id first to check the chip is there", cannot work:

the very first frames at boot must be the blind write enable plus WRSR that sets QE= 1 in status register 2 (QE disables the `/HOLD#`/`/WP#` pin functions on this part) and clears the protection bits in status register 1, then the Global Block Unlock (`0x98`). Only after that does the chip answer, and only then does the firmware read the JEDEC id to set its *presence* flag (first byte `0xEF` = Winbond). If the id is wrong the flag stays low and every later storage opcode fails fast with `'E'` instead of hammering timeouts.

The timeouts themselves changed nature in the port. The hardware counted cycles (25 million, ≈ 620 ms); the firmware counts *time* on `rdcycle`, because a C loop iteration has no fixed duration: 2 ms per SPI byte transfer (a transfer at 633 kHz takes 13 μs, so this only trips if the engine is dead), 700 ms on the WIP poll (a sector erase can take 400 ms), 200 ms per payload byte from the ESP32 (a truncated command aborts without acknowledging, and the ESP32's own retry takes over). A dead SPI or an absent chip therefore costs a bounded few seconds at boot, not a hang.

9.5 The bus side: S7 commands in, S8 frames out

The two slaves that replaced `flash_ctrl` are deliberately dumb. S7 is a second instance of the same UART generic used for the characters, on the same FPGA pins (72/20) the hardware block used, so the ESP32 did not change a wire or a byte of its protocol. S8 is the generic SPI byte engine: one byte per transfer, chip select held by a register bit across bytes so a multi byte W25Q frame is just a CS assert, n TXDATA writes, a CS release. Its `done` flag is persistent, cleared by the next TXDATA write rather than by reading STATUS, for the reason of Section 3.4; and the S7 UART is the peripheral that motivated the 16 entry receive FIFO, because a 16 byte append payload arrives back to back at 115200 baud while the CPU may be mid way through a display chunk or a spectrum copy.

Table 10: Generic SPI engine register map (slave S8, base `0x3800_0000`).

Offset	Dir	Function
`0x00`	R	RXDATA, the last received byte
`0x04`	W	TXDATA, write a byte and start a full duplex transfer (clears `done`)
`0x08`	W	CTRL, bit 0 = chip select (1 asserts CS low, held across bytes)
`0x0C`	R	STATUS, bit 0 = busy, bit 1 = done (non destructive read)

One behavioural addition came free with the port: the firmware applies the settings it loads. At boot, and again on every `S` save, it parses the 32 byte blob and immediately adopts the debug flag (which gates all the `$` diagnostic traffic, so verbosity is now a dashboard switch instead of a recompile), the TFT theme colours and the view flags; a settings epoch counter ticks on every apply, and the display manager redraws when it sees it change (Section 10). The boot also prints a one line summary on the diagnostic channel (`$NOR ok id=EF hd=... lk=0`), which replaced a whole debugging session's worth of guessing about the state of the chip.

10 The TFT display: a fourth SPI and a mini dashboard

Until this point the only way to see what the master was doing was the UART console or the web dashboard, both of which need a laptop. The last addition is a 2 inch IPS panel (a GMT020-02-7P, ST7789 controller, 240 × 320) wired straight to the FPGA, on which the soft core draws its own mini dashboard. It needs five wires, on five spare FPGA pins (27 to 31): SCL, SDA, DC, RST and CS.

10.1 The `spi_display` slave

The panel is driven by a fourth SPI master, and it is the simplest of the four because the ST7789 in this wiring never talks back: `spi_display.vhd` is transmit only (no MISO pin at all). It runs SPI mode 3, which is what the panel expects: SCK idles *high*, the data line changes on the falling edge and the panel samples it on the rising edge, MSB first. The prescaler is ÷6, so SCK is 40.5/6 = 6.75 MHz, by far the fastest SPI in the design; it can afford to be, because the panel sits centimetres from the FPGA on its own wires, with none of the shared ground drama that capped the NOR at 633 kHz. The three panel control pins are register bits: DC (the ST7789 command/data select), RST (the panel reset) and CS. Being the newest slave, it was written with the Section 3.4 rules from the first line: ack held while the strobe is up, and a busy flag whose read is non destructive.

Table 11: `spi_display` register map (slave S9, base `0x5800_0000`).

Offset	Dir	Function
`0x00`	W	TXDATA: write a byte and start the shift (ignored while busy)
`0x04`	W	CTRL: bit 0 = DC (0 command, 1 data), bit 1 = RST pin, bit 2 = CS assert
`0x08`	R	STATUS: bit 0 = busy (non destructive read)

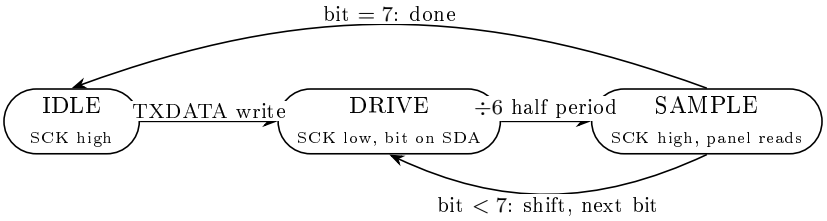

Figure 18: The `spi_display` shifter. A TXDATA write loads the byte and pulls SCK low with the MSB already on SDA; each bit then spends half a ÷6 period low (data driven) and half high (panel samples, mode 3); after the eighth bit SCK parks high, which *is* the mode 3 idle, so byte boundaries need no extra states.

10.2 Bring up: the ST7789 init sequence

After releasing the panel reset (RST low 20 ms, then high, then 150 ms of patience), the firmware sends the seven command init that the datasheet prescribes, each with its mandated delay: SWRESET (150 ms), SLPOUT (120 ms), COLMOD = `0x55` (16 bit RGB565 pixels), MADCTL = `0x60` (landscape, so the 240 × 320 panel becomes 320 × 240), INVON (this IPS panel wants inverted polarity, without it colours come out negative), NORON and DISPON. From then on drawing is just three commands: CASET and RASET to declare a rectangular window, RAMWR followed by $w \times h$ RGB565 pixels.

10.3 `display_manager`: drawing without blocking

A full screen is $320 \times 240 \times 2 = 153{,}600$ bytes, which at 6.75 MHz is ≈ 180 ms of solid pushing. The main loop cannot disappear for 180 ms: the tone player would stutter and, worse, the S7 FIFO would overflow mid append. So the display manager never draws synchronously. Drawing requests become *jobs* (fill this rectangle, write this text) in a 48 entry queue, and each `display_tick()` call executes at most one small chunk of the current job: 64 pixels of a fill, or one glyph of a text (a 5×7 font, scaled in integer steps). A chunk is 128 to 384 SPI bytes, 150–450 μs, which is invisible next to the 10.9 ms audio frame; the screen simply assembles itself over a few dozen loop passes.

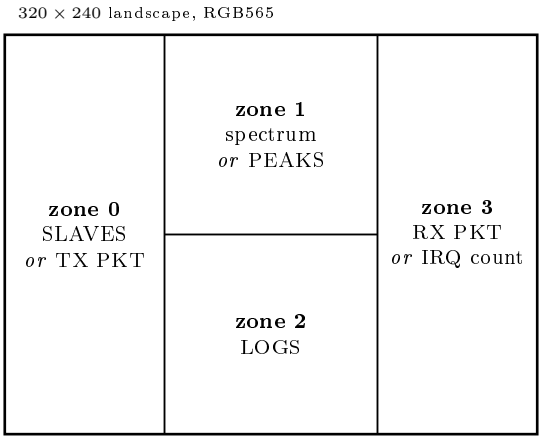

Figure 19: The four fixed zones of the on screen dashboard. What each zone shows is chosen by the view flags byte of the NOR settings (each zone has a priority list, first enabled flag wins), and the three theme colours come from the same settings, so the dashboard restyles the TFT remotely; a save bumps the settings epoch and the display redraws from scratch.

The zone contents are all data the firmware already had. SLAVES renders the health mask of Section 12 as a green/red OK/KO list, one row per bus slave plus the NOR presence flag. The spectrum view draws one bar per FFT bin in the band the decoder cares about (bins 16 to 55), straight out of the same `g_sdram[]` snapshot the decoder reads, with the protocol bins in the accent colour, so the panel shows the tones arriving in real time; PEAKS is its numeric twin (the two strongest bins with their frequency and the symbol they would decode to). RX and TX show the last decoded character and counters from the emitter; IRQ shows the accelerator frame counter; LOGS is a six line ring of short firmware events (NOR appends, clears, settings saves). Beyond the init delays, the manager is a flat round robin: one zone's content is marked dirty every 1.5 s and redrawn in chunks, and a settings epoch change marks everything dirty at once.

11 Signal integrity and decoupling

None of the problems in this section showed up in simulation. All of them showed up on the bench, and the fixes are a mix of a few capacitors, a slower clock, and firmware retries. The common thread is that the W25Q64 sits on a breakout board connected to the FPGA by about 10 cm of Dupont jumper wires, which is a poor environment for both power delivery and signal edges.

11.1 Brownout during sector erase

To erase a 4 KB sector the chip runs an internal charge pump that draws around 80 mA in peaks for about 30 ms (typical), up to 400 ms worst case. To program one record the draw is lighter, around 25 mA for under 3 ms. The supply reaches the chip through the jumper wires, so any current spike shows up as a voltage droop at the chip.

The droop over a decoupling capacitor follows $\Delta V = I\,t/C$. With no capacitor at all (only the tens of pF of the wire), the 80 mA erase current collapsed the 3.3 V rail almost instantly. The chip browned out mid erase, lost its write enable, silently rejected the program that followed, and the firmware saw a success that had not happened.

Adding a 1 μF ceramic across VCC and GND at the chip gives

$$\Delta V = \frac{0.080 \times 0.030}{1 \times 10^{-6}} = 2.4 \text{ V},$$

which still drops the rail to 0.9 V, below the 2.7 V minimum, so log writes started working but settings saves (erase plus program) still failed. Adding a second capacitor of 10 μF in parallel, for

11 µF total, gives

$$\Delta V = \frac{0.080 \times 0.030}{11 \times 10^{-6}} = 0.22 \text{ V},$$

so the rail only drops to 3.08 V, safely above 2.7 V, and the brownout is gone. The two values cover different bands: the 1 µF has low ESR/ESL and handles the fast SPI edges, the 10 µF holds up the slow charge pump current. The minimum capacitance to stay inside a 0.6 V budget is $C \geq I\,t/\Delta V = 0.080 \times 0.030/0.6 = 4\,\mu\text{F}$, so 11 µF has comfortable margin.

11.2 SPI clock too fast for the wiring

Separately, the first SPI clock of 5 MHz was too fast for 10 cm of jumper with a single ground return. Long writes came out truncated: a 16 byte record would arrive with byte 10 onward stuck at `0xFF`. I lowered the clock in steps and watched the first attempt success rate: 5 MHz was bad, 2.5 MHz better, 1.27 MHz mostly good, and 633 kHz (divide by 64) almost always good. At 633 kHz a bit lasts 1.58 µs, long enough that the ringing on the wire settles well before the chip samples.

11.3 Retry and verify in firmware

Even at 633 kHz an occasional write still fails (roughly 1 in 100 on settings, rarer on log records). The firmware does up to 5 attempts: send, wait for the acknowledgement, read the slot back, compare all 16 bytes, retry if they differ. A failed attempt burns one ring buffer slot, but the SCAN at boot ignores those because their type byte is not in the valid set, so nothing is lost permanently. Table 12 is the measured reliability after all three fixes.

Table 12: Measured first attempt reliability after 633 kHz SPI, 11 µF decoupling and firmware retry/verify.

Operation	First attempt	After up to 5 retries
log record write (16 B)	≈ 95%	≈ 100%
settings save (32 B, with erase)	≈ 85%	≈ 100%
read back of a record	100%	100%
JEDEC ID read at boot	100%	100%

A separate hardware lesson, worth recording: an indicator LED that I had wired with its anode straight to an FPGA pin and its cathode to the common ground, with no series resistor, was injecting tens of mA into the ground return. That current, through the impedance of the shared ground wire, moved the ground reference enough to corrupt the microphone ADC readings (the FFT then "heard" tones that were not there) and to disturb the UART and SPI at the same time. Removing the resistorless LEDs fixed all of it at once. The rule is to always put a series resistor (330 Ω to 1 kΩ) on an LED driven from a logic pin.

12 Soft core firmware

The firmware on the PicoRV32 is still compact: seven C files and a startup assembly file, compiled with the RISC-V GCC toolchain for `rv32imc` at `-O1`. The linker script places code and constants in the 32 KB ITCM at `0x0200_0000` and data, BSS and stack in the 16 KB DTCM at `0x0100_0000`; both are CPU local block RAM, off the Wishbone. The build ends with a small script that flattens the binary into `ram32.hex`, one 32 bit word per line, which is the format the Gowin PicoRV32 IP wants as its ITCM initialisation file; the firmware therefore travels inside the bitstream, and changing the firmware means regenerating the IP. (My first attempts handed the IP the Intel HEX

file that GCC produces, which it accepts silently and loads as garbage; the flat format is the one that works.)

The structure is a single main loop with no operating system and, deliberately, no interrupts (Section 5.2): each pass calls the transmit side (`emit_capture`, `emit_player_tick`), the receive side (`decode_step`), the NOR command dispatcher (`nor_poll`, Section 9), the bus health scan (`health_tick`, below), the display renderer (`display_tick`, Section 10) and, when the debug flag is set, the diagnostic dumps. Every function is non blocking, so each loop pass is short and every flag gets polled within microseconds; the two costliest passes, a 512 word spectrum copy and a display chunk, are each a few hundred microseconds and cannot happen back to back often enough to matter. The debug flag itself is no longer a compile time constant: it is bit `b[19]` of the NOR settings, loaded at boot and re-applied live on every save, so verbosity is toggled from the web dashboard without touching the toolchain.

12.1 Transmit: from UART character to tone pair

`emit_capture` peeks the character UART (S6): if a received byte is one of the protocol characters (`a`–`d`, `A`–`D`) it is pushed into a 16 entry software queue. `emit_player_tick` is the player, a three state machine. In `P_IDLE` it pops a character, maps it to the tone pair (the master carrier at 2014 Hz in the low half word, the data tone in the high half word), writes the pair to the PWM start register and moves to `P_TONE`; after 144 ms it writes stop and rests in `P_SIL` for 400 ms. The characters `d`/`D` are a special *warm up* symbol: the same EOF tone pair, held for 300 ms instead of 144 (Section 14).

The player is also where the carrier sense lives. The decode side keeps a flag, *channel busy*, true whenever the slave carrier has been heard in the last 3 seconds. While the flag is up, `P_IDLE` holds the queue and emits nothing; and if the flag rises in the middle of a tone, `P_TONE` stops the PWM at once and yields. This is the half duplex rule of the protocol enforced at the lowest level that can react quickly.

12.2 Receive: poll, copy, decode

`decode_step` calls `read_sdram_tick`, the polling driver of Section 5.2: it reads `fill_cnt` (S0 offset `0x90`), compares it with the last value seen and, only when it changed, copies the 512 bins from the BSRAM snapshot into a RAM array and raises a ready flag. Note that the poll target is the counter, never the BSRAM itself: one bus read per loop pass answers "is there a new frame?", and the 512 word copy is paid only when the answer is yes (a counter also needs no clearing, Section 5.2). The decoder (`decoder.c`, the C port of the VHDL decoder, Section 6) then runs once per frame on that array. Of its output, only the uppercase characters `A`–`C`, the symbols riding the *slave* carrier, are forwarded to the ESP32; the lowercase ones are the master hearing its own transmission through its own microphone, useful as a loopback check but not protocol input. Independently of the full decode, a dedicated check on bin 27 alone (the slave carrier) refreshes the channel busy timestamp, so carrier sense works even when no complete symbol is recognisable.

12.3 The health scan

With ten slaves on the bus, "the system stopped working" has ten boring explanations before any interesting one, so the firmware checks them itself. Every 15 seconds `health_tick` reads one *safe* register from each slave, applies a sanity check to the value (the fetch FSM status must show a legal burst and index, a PWM data read must return its wired zero, a UART status must have its reserved bits at zero), and assembles a bit mask, S0 through S8. Two slaves need indirection: the ADC SPI (S1) is never read directly, because its data register only acks when a sample is ready and the accelerator, which outranks the CPU, consumes every sample, so its health is inferred from the accelerator's sample counter advancing; and the FFT chain is judged the same way from the frame counter. The mask is OR-ed with the previous scan before reporting, so a single dirty

read cannot flag a healthy slave, and the result goes out on the diagnostic channel as a `$HLT` line that the ESP32 forwards to the dashboard, which renders the per slave OK/KO lights; the same mask feeds the SLAVES zone of the TFT. The failure mode is deliberately informative: if a slave stops acking, the CPU hangs on that read, the `$HLT` heartbeat disappears, and the dashboard marks the system as lost, while the first scan after boot prints a probe marker before each first touch (`$HP s2`, `$HP s3`, ...), so the last marker printed names the slave that killed the CPU. That trick found the GPIO ack bug of Section 3.4 in minutes.

12.4 The diagnostic channel

Printable diagnostics share the one character UART with the protocol traffic, so they are framed: every diagnostic line starts with `$` and ends with a newline, and the ESP32 forwards anything between those two verbatim to the USB console instead of feeding it to the protocol parser. Behind the settings debug flag the firmware emits, on this channel, a once per second system line (loop count, queue depth, the three hardware counters of Table 5), a per decode line with the magnitudes of the five protocol bins, and every three seconds a dump of all 512 bins from the snapshot. That last dump is how I distinguished "the FFT is not running" from "the FFT runs but the CPU reads garbage" more than once. Two lines ignore the flag because they are cheap and load bearing: the one shot `$NOR` boot summary and the 15 second `$HLT` heartbeat, which the ESP32 always forwards to the dashboard but echoes on the USB console only when debug is on, so a quiet console stays quiet.

13 ESP32 firmware

The ESP32 firmware is split into small modules. `uart_reg` is a register level driver for the ESP32 UART1 and UART2 peripherals, with no Arduino HardwareSerial layer: UART2 (ESP32 pins 27/14) is the character link to the FPGA's S6 UART, UART1 (pins 32/33) is the storage command link, which now lands on the FPGA's S7 UART and is served by the soft core (the ESP32 never noticed the hardware block behind it being swapped for firmware). `flash_link` performs the high level flash operations against the FPGA controller, with retry and verify and a 60 record RAM cache for the dashboard. `fpga_uart_link` is the codebook layer that turns decoded characters into protocol patterns and drives the speaker through the FPGA. `protocol` is the acoustic protocol state machine (presence, retries, abort, registration). `web_link` is the WebSocket client and JSON layer for the dashboard; it also drives two status LEDs (Wi Fi up on GPIO 25, WebSocket up on GPIO 26) at register level, through the `GPIO_OUT_W1TS/W1TC` set/clear registers rather than `digitalWrite`. One power lesson belongs here: with the radio at full transmit power the ESP32's current peaks tripped its own brownout detector on a weak supply, so the firmware caps the Wi Fi transmit power at 11 dBm; the real cure is a solid 5 V feed, but the cap keeps the board alive on marginal ones.

13.1 Register level UART

The two UART links to the FPGA are driven by writing the UART peripheral registers directly (the FIFO, CLKDIV, CONF0 and STATUS registers of UART1 and UART2), not through HardwareSerial. The baud divider for 115200 from the 80 MHz APB clock is

$$\text{int} = \left\lfloor \frac{80\,000\,000}{115200} \right\rfloor = 694, \qquad \text{frac} = \left\lfloor \frac{80\,000\,000 \times 16}{115200} \right\rfloor \bmod 16 = 7,$$

so the CLKDIV register is $694 \,|\, (7 \ll 20) =$ `0x7002B6`, giving $80\,000\,000/(694 + 7/16) = 115{,}208$ baud, 0.007% off. A free running 1 MHz hardware timer (TIMG0) gives microsecond timeouts for the UART reads without using `micros()`.

13.2 Flash link: retry and verify, and a defensive read

When the dashboard sends a note, `web_link` parses the JSON and calls `flash_log_append`, which builds the 16 byte record, sends opcode `L` plus the bytes over UART1, waits for the `K` acknowledgement, reads the head back with `H`, reads the just written slot back with `G`, and compares all 16 bytes; on a mismatch it retries, up to 5 times. The record is also pushed into the RAM cache so the dashboard still shows it even if a later NOR read misbehaves. The settings save works the same way against the 32 byte sector. The settings read, `flash_get_settings`, validates before returning: the magic byte must be `0xA5`, the retry count must be between 1 and 30, and the name must be printable ASCII; if any check fails it returns safe defaults, so the dashboard never shows a garbled name or an absurd count even when the NOR has a half written sector.

The settings blob grew with the display: alongside name, auto presence interval and retry count, the dashboard now edits the debug flag, the three TFT theme colours (sent to the browser as hex strings, stored as raw RGB bytes) and the view flags that choose what the TFT zones show. The ESP32 is a pure courier for these fields, it stores and forwards them without interpreting anything except the debug flag, which it also applies to itself: the `$HLT` health lines from the FPGA always reach the dashboard, but they are echoed on the USB console only when debug is on.

14 The acoustic protocol

The protocol uses an alternating bit codebook. A pattern is two or three symbols, with the rule that two consecutive symbols must differ; the decoder then triggers on a *change* of tone, which is easy to see, rather than on silence, which is hard to measure over a noisy channel. There are six symbols: lowercase `a`, `b`, `c` (bit 0, bit 1, end of frame) ride the 2014 Hz master carrier, and uppercase `A`, `B`, `C` ride the 2472 Hz slave carrier. The master sends lowercase and the slave sends uppercase, so each side knows who spoke.

A message is identified by its first bit and its length, while the carrier gives the direction; the same bits therefore mean two different messages on the two carriers. Table 13 is the whole codebook. Each pattern is framed by EOF tones (`c` or `C`) at both ends.

Table 13: The alternating bit codebook. A pattern (its first bit and length) plus the carrier it rides on selects one message; the same pattern means different things on the two carriers, because the carrier says who is speaking. On the master carrier the data bits are the tones `a`/`b`, on the slave carrier `A`/`B`.

Pattern	(first, length)	Master carrier (to slave)	Slave carrier (to master)
`01`	(0, 2)	presence request	presence: yes
`10`	(1, 2)	assign id (= 1)	presence: no
`010`	(0, 3)	abort	registration request
`101`	(1, 3)	—	acknowledge

14.1 On the air: warm up, redundancy, and detection by change

What actually goes on the air for one message is more than the two or three data symbols, because the channel is a cheap speaker, a cheap microphone, room reverberation, and an automatic gain control on the receiving capsule. Three field driven additions wrap the pattern (Figure 20):

- a *warm up* symbol opens every transmission: the EOF tone held for 300 ms instead of 144 (the ESP32 sends the special character `d`, Section 12). Its job is to let the receiver's microphone AGC settle on a long, harmless tone; on the receiving side it is just an EOF, so it flushes an empty pattern and carries no data;

- the EOF framing is sent as a *burst of four* at both ends of the pattern, because single symbols get lost on this channel and an EOF is the one symbol whose loss breaks the framing rather than one bit;
- every symbol is paced: 144 ms of tone, 400 ms of silence. The long gap is the symbol separator; with a short one, the reverberation tail of a tone runs into the next and two equal bits fuse into what sounds, and decodes, like one.

The receiver mirrors this with *detection by change*: a decoded character is accepted as a new symbol only if it differs from the last accepted one. This is what the alternating codebook buys: inside a pattern two consecutive data bits always differ, so a repeated character can only be the FPGA decoder reporting the same tone across several FFT frames (each tone spans many 10.9 ms frames) or reverberation holding a tone up, and both are correctly collapsed into one symbol. The four EOFs of the framing burst likewise collapse into a single EOF event. A partial pattern with no new symbol for 2 seconds is discarded, so a truncated frame cannot poison the next one.

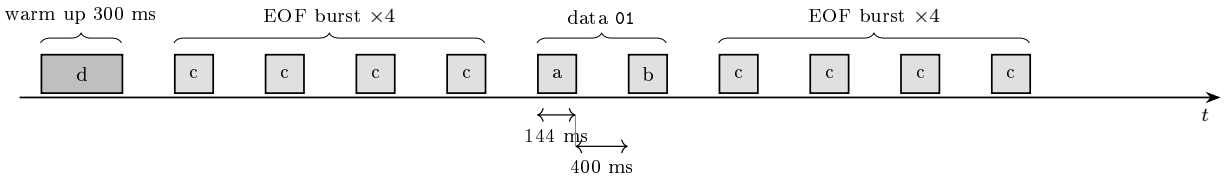

Figure 20: One transmission on the air (here the presence request, pattern `01`). Every box is a two tone burst (carrier + data tone); the darker, longer first box is the warm up EOF that settles the receiver's AGC. The receiver collapses each EOF burst into one EOF event by detection by change, so what reaches the protocol layer is exactly `c 0 1 c`.

14.2 Half duplex, collisions and retries

The channel is half duplex, so only one side transmits at a time, and the master enforces it at two levels. On the soft core, the channel busy flag (Section 12) holds the tone player whenever the slave carrier has been heard in the last 3 seconds, and aborts a tone already in progress; this is the fast reflex. On the ESP32, the link layer waits before *every* byte it sends for the channel to have been quiet for 1 second (giving up after 15 seconds, so a noisy decoder cannot block transmission forever); this spaces whole patterns rather than single tones. If the master is emitting and the slave carrier appears, the tone in progress stops and the master yields, so a collision resolves to one talker.

Retries are handled by the ESP32, and they exist because the channel loses symbols routinely, not exceptionally. A presence request with no answer within 8 seconds is repeated, up to a configurable number of attempts (default 6, stored in the flash settings), after which the master gives up and tells the dashboard. The registration SET is retransmitted on a slow randomised timer (30 to 40 seconds) until the acknowledgement arrives; the randomisation avoids the two nodes locking into the same retry rhythm. Receiving is idempotent on both sides, so a duplicate caused by a retry does no harm.

15 Power supply

Power enters as a single 12 V supply rated 2 A. A fuse and a series Schottky diode protect against overcurrent and reverse polarity, then an LM2596 based DFRobot DFR1202 buck module steps the 12 V down to 3.3 V at up to 0.5 A for all the digital parts (the FPGA board, the ESP32, the NOR flash, the ADC). Input and output of the buck are decoupled with 100 μF and 100 nF capacitors. The 12 V rail is kept separate from the buck and feeds the two high current loads directly: the 6 W power LED and the speaker stage, each switched by its own low side MOSFET

from a logic pin. Keeping the LED and speaker on the raw 12 V rather than on the 3.3 V buck keeps their switching current off the supply that feeds the sensitive ADC and flash.

16 What worked and what did not

The parts that worked end to end are the bus and its address map, the SDRAM controller once the refresh interval and the 1.54 ns clock phase shift were set, the 512 point FFT with the parabolic peak decoder and its ±3 neighbourhood, the alternating bit codebook (helped a lot by giving each direction its own short sub codebook for the most repeated message, the presence request), the BSRAM snapshot path that finally gave the CPU clean spectra, the NOR flash controller after the decoupling and clock work, the WebSocket dashboard, and the register level UART driver on the ESP32.

A few things were tried and dropped, or moved. The early design had a generic DMA between the ADC and SDRAM with the CPU programming descriptors; it spent too many CPU cycles per window, so I folded the gather and FFT launch into the streaming accelerator. The first CPU read path to the spectrum, chasing the fetch FSM's live burst buffer over the bus, lost the race and produced torn spectra; the hardware BSRAM snapshot replaced it (Section 5.2). The FFT done interrupt is wired to the core and the ISR plumbing exists, but polling won on simplicity and the IRQ now only triggers hardware. The tone emitter and the tone decoder both started as RTL state machines and were both ported to firmware once the soft core was up; the RTL versions are still in the tree, disabled, as references. The flash controller followed the same path last: once the bus rules of Section 3.4 held, `flash_ctrl` was retired and its state machine re-emerged as `norflash.c`, behind the same ESP32 protocol. Driving the UART pin to the ESP32 through a three way AND of banner, hardware decoder and CPU UART broke the line in subtle ways, so the pin now belongs to the CPU's UART alone. The ±1 decoder neighbourhood was too strict and was widened to ±3. The plain HardwareSerial on the ESP32 worked but did not show register level work, so I replaced it with the direct register driver.

What still wobbles is mostly the wiring. The W25Q breakout is on Dupont jumpers, so the SPI clock cannot realistically go above 633 kHz; on a proper board with a ground plane it would run far faster with no retries. The decoupling is 1 plus 10 μF rather than the more orthodox 100 nF plus 10 μF, because that is what I had on hand. And the very first status register read after power up sometimes returns `0xFF` before the chip is ready, which makes the firmware print a false "chip locked" warning that clears on the next read.

The migration that closed the loop is done: the flash protocol lives in the soft core (S7 in, S8 out, Section 9), and the freed debug pins now drive a TFT that shows the system state without a laptop (Section 10). What remains is cleanup debt: the fetch FSM still carries its bring up name `tst_` in the RTL, and the retired blocks (`flash_ctrl`, the hardware decoder and emitter) still sit in the project file as disabled references waiting to be dropped.

17 Conclusions

The master node is a working integration of a custom SoC on the Gowin GW2AR-18, an acoustic receive pipeline from analog microphone through a 12 bit ADC, a 512 point FFT and a tone decoder, a NOR flash store whose protocol runs in the soft core, a TFT dashboard drawn by the same core, and an ESP32 that runs my AUSP protocol on top and bridges it to a web dashboard. The single most useful lesson was that for the mixed signal parts the datasheet number is where the problem starts, not where it ends: the simple relation $\Delta V = I\,t/C$ explained a "flash write succeeds but the data is not there" bug that had looked like a firmware fault for days.

A Quick reference

Table 14: Key numbers used in the design.

Bus / CPU clock	40.5 MHz
SDRAM clock	40.5 MHz; die on phase shifted `clkoutp` (+1.54 ns, 22.5°)
SDRAM refresh	one AUTO REFRESH every 7.81 μs
SDRAM traffic	bursts of 26 words, one row per burst; 20×26 per frame
ADC sample rate	46.875 kHz (40.5 MHz / 864)
FFT	512 point, bin width 91.6 Hz; one frame every 10.92 ms (91.6 Hz)
Tone bins	22, 27, 32, 39, 50 (2014 / 2472 / 2930 / 3571 / 4577 Hz)
Spectrum snapshot	BSRAM at S0 `0x4800_1000`; poll `fill_cnt` at `0x90`
Tone pacing	144 ms tone + 400 ms silence; warm up EOF 300 ms
PWM carrier	speaker 158.2 kHz (40.5 MHz / 256); LED 105.5 kHz (27 MHz / 256)
NOR SPI clock	633 kHz (divide by 64), via slave S8
ADC SPI clock	750 kHz (divide by 54)
Display SPI clock	6.75 MHz (divide by 6, mode 3, TX only)
UART baud	115200 (divide by 352); RX FIFO 16 deep
W25Q64 JEDEC ID	EF 40 17
Sector erase / page program	up to 400 ms / up to 3 ms
Flash timeouts (firmware)	2 ms / byte, 700 ms WIP poll, 200 ms / payload byte
Decoupling at the flash	1 μF + 10 μF
Log record / settings	16 B / 32 B v2 (marker `0x5A`), 512 ring slots
Health heartbeat	`$HLT` every 15 s; TFT zone refresh every 1.5 s
Protocol retries	presence: every 8 s, 6 tries (configurable); SET: 30–40 s
Carrier sense	soft core: 3 s hold; ESP32: 1 s quiet per byte, 15 s cap